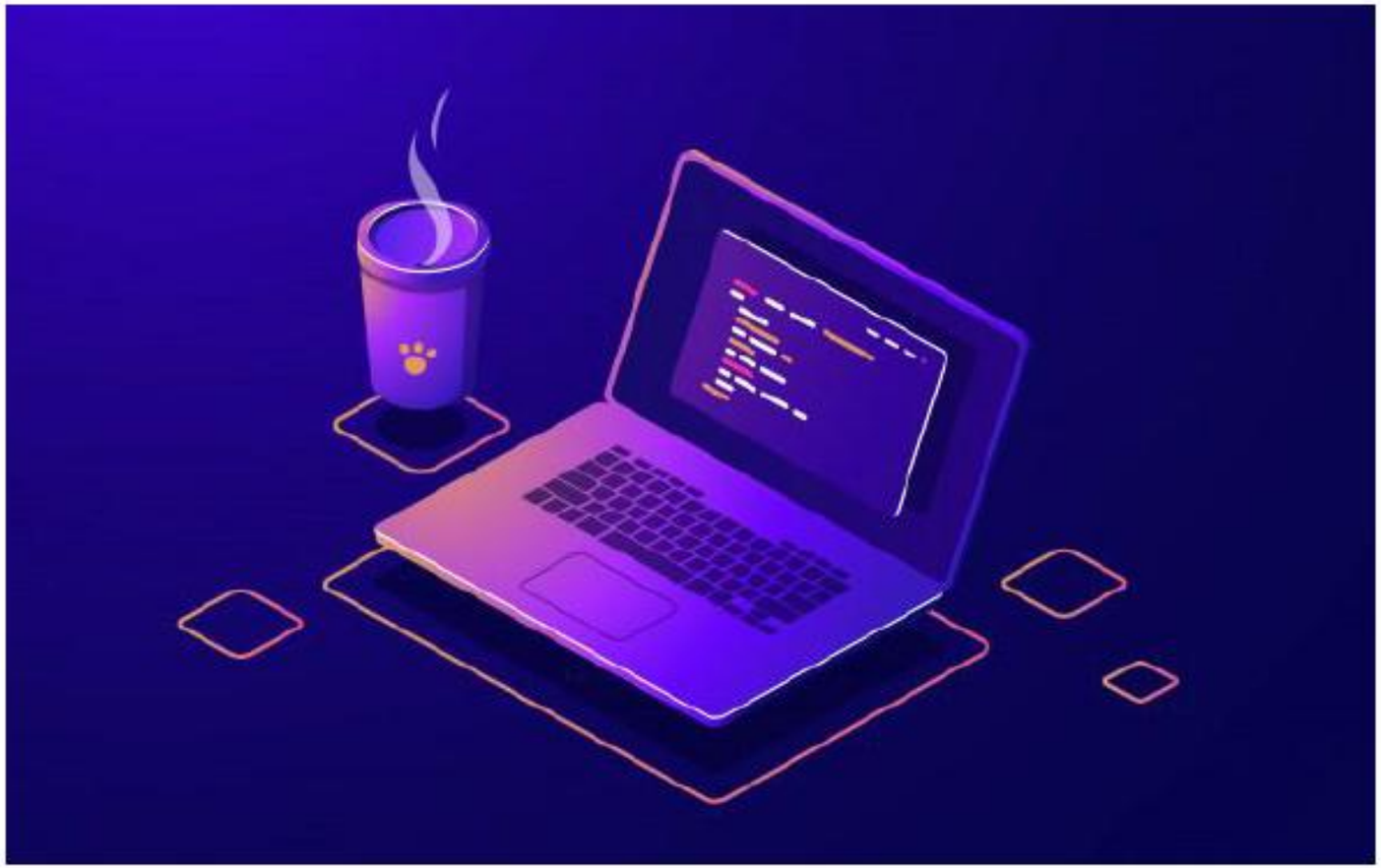


PROGRAMACIÓN

TEMA 8:

ESTRUCTURAS DE ALMACENAMIENTO

Gestión de Datos en Memoria

ÍNDICE

- Introducción: datos en memoria y enfoque moderno
- Arrays como punto de partida (mínimo imprescindible)
- Framework de colecciones y genéricos
- Colecciones núcleo: List, Set y Map (elección y patrones)
- Contratos y ordenación (imprescindible para colecciones “reales”)
- Transición a funcional: clases anónimas y expresiones lambda
- Operaciones funcionales directas sobre colecciones
- Streams y Collectors para tratamiento moderno de colecciones

Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Estructuras vs operaciones (colección vs stream)

¿Qué es gestionar datos en memoria?

En un programa Java, “gestionar datos en memoria” significa diseñar y usar estructuras para:

1. **Representar entidades** (modelo)
   - Usuario, Producto, Pedido, Evento/Log… suelen ser **objetos (POJOs).**
2. **Mantener colecciones** de esos objetos durante la ejecución
   - El tamaño es variable, y el programa necesita insertar, eliminar, buscar, ordenar, agrupar.
3. **Acceder y transformar datos** con garantías y coste controlado
   - Correctitud (duplicados, orden, consistencia).
   - Rendimiento (tiempo y memoria) según operaciones dominantes.

Qué implica técnicamente:

- Los datos residen en el heap: referencias a objetos, recolección de basura (GC) y coste de crear/descartar objetos.
- Elegir estructura de datos no es “estético”: depende de requisitos de:
  - **Orden** (¿mantener orden de inserción?, ¿ordenar por clave/campo?)
  - **Unicidad** (¿permito duplicados?)
  - **Acceso** (¿por índice?, ¿por clave?)
  - **Complejidad** (coste típico O(1), O(n), O(log n)).

Ejemplo mínimo (modelo + almacenamiento):

```
record Pedido(String id, String cliente,
double total) {}
List<Pedido> pedidos = new ArrayList<>();
// Secuencia, permite duplicados
Set<String> clientesUnicos = new HashSet<>();
// Unicidad
Map<String, Pedido> porId = new HashMap<>();
// Acceso por clave (id)
```

- **Colecciones** = “estructura” (dónde guardo los datos).
- **Lambdas/Streams** = “procesamiento” (cómo filtro, ordeno, agrupo y transformo esos datos).

Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Estructuras vs operaciones (colección vs stream)

Ejemplo real: Almacenamiento dinámico de datos

```
List<String> usuarios = new ArrayList<>()
usuarios.add("Ana");
usuarios.add("Luis");
usuarios.add("Marta");
```

¿Qué representa este código?

- Un **conjunto de datos homogéneo** (todos del mismo tipo: `String`)
- Una **estructura única** que agrupa múltiples valores
- Datos que pueden **aumentar o disminuir durante la ejecución**

Operaciones implícitas sobre el conjunto

Aunque solo vemos `add(...)`, una estructura de este tipo permite:

- **Inserción** de nuevos elementos
- **Recorrido** secuencial de todos los datos
- **Acceso** a elementos individuales
- **Modificación** de valores existentes
- **Eliminación** de elementos

Es decir, no solo almacena → **permite operar sobre el conjunto**

Modelo de uso en un programa

Este tipo de estructura se utiliza cuando:

- El número de elementos **no se conoce a priori**
- Los datos pueden **cambiar durante la ejecución**
- Se requiere **tratar los datos como grupo**

Ejemplos típicos:

- Lista de usuarios conectados
- Productos en un carrito
- Resultados de una consulta

Comparación con variables independientes

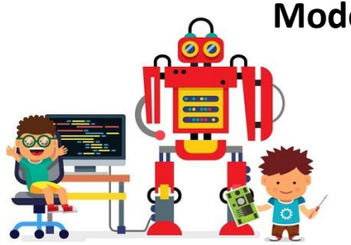
```
String u1 = "Ana";
String u2 = "Luis";
String u3 = "Marta";
```

Limitaciones:

- No se pueden recorrer fácilmente
- No se pueden procesar de forma conjunta
- No escalan si el número de elementos crece

Propiedades observables (sin implementación)

- Mantiene una **colección de elementos relacionados**
- Permite **crecimiento dinámico**
- Facilita el **tratamiento uniforme de los datos**



Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Estructuras vs operaciones (colección vs stream)

Operaciones necesarias sobre un conjunto de datos

En memoria, los datos rara vez se usan como valores aislados. Normalmente aparecen como conjuntos (usuarios, pedidos, productos) y el programa necesita aplicar operaciones sistemáticas sobre ese conjunto.

1. **Operaciones típicas sobre un conjunto de datos (definición operativa)**
   - **Inserción:** incorporar elementos nuevos al conjunto.
   - **Recorrido:** visitar todos los elementos para mostrar/calcular/validar.
   - **Actualización:** localizar un elemento y modificar su estado/atributos.
   - **Filtrado:** seleccionar subconjuntos según una condición booleana.
   - **Ordenación:** establecer un criterio de orden para presentación o procesamiento.
2. **Consecuencia de diseño: separar "dónde están" de "qué hago con ellos"**
   Estas operaciones introducen una separación natural en el código:
   - **Estructura (almacenamiento):** mantiene el conjunto de elementos.
   - **Operaciones (procesamiento):** aplican transformaciones y consultas sobre ese conjunto.
3. **Resultado esperado del tema (sin entrar aún en streams)**
   Al finalizar el tema podremos:
   - Elegir y usar una estructura adecuada para almacenar datos (colecciones).
   - Aplicar operaciones de procesamiento sobre esos datos de forma moderna (lambdas/streams) sin mezclar accidentalmente "almacenamiento" y "lógica".

Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Imperativo vs Declarativo. Por qué existe *streams*

Dos Estilos para Procesar Colecciones: Imperativo y Declarativo.

Cuando trabajamos con una colección (p. ej., pedidos), casi siempre aplicamos operaciones como filtrar, transformar u obtener resultados agregados. Java permite expresar ese procesamiento de dos maneras principales. La diferencia no es “de estilo”: afecta a **cómo se estructura** el código, **cómo se compone** y **qué partes quedan acopladas**.

1. **Enfoque imperativo (control explícito del proceso)**

   En el estilo imperativo el programador describe cómo se recorre la colección y en qué puntos se aplica la lógica. El bucle controla el flujo y la condición filtra; la acción se ejecuta dentro del propio recorrido. Es correcto, directo y a veces imprescindible, pero mezcla mecánica de iteración con reglas del problema.

```
for (Pedido p : pedidos) {
    if (p.getTotal() > 100) {
        System.out.println(p);
    }
}
```

   - El “cómo” domina: iterar → comprobar → actuar.
   - La lógica queda embebida en el bucle (difícil de reutilizar o componer).

2. **Enfoque declarativo (expresión de intención)**

   En el estilo declarativo se expresa qué operación se quiere realizar sobre los datos. La librería se encarga del recorrido y el programador define la lógica como operaciones encadenadas (pipeline). Esto separa mejor “datos” de “operaciones” y facilita composición.

```
pedidos.stream()
       .filter(p -> p.getTotal() > 100)
       .forEach(System.out::println);
```

   - El “qué” domina: filtrar → consumir.
   - El recorrido y el flujo quedan encapsulados en la API.

Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Imperativo vs Declarativo. Por qué existe *streams*

Dos Estilos para Procesar Colecciones: Imperativo y Declarativo.

3. **Por qué existe Streams (motivación técnica)**

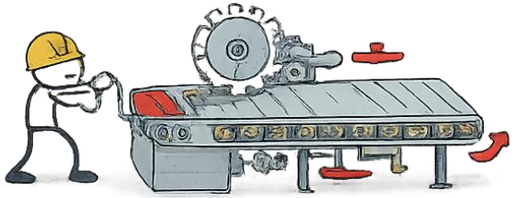
   Streams introduce un modelo estándar de procesamiento de colecciones basado en operaciones componibles. En lugar de reescribir bucles para cada caso, se reutilizan operaciones típicas:

   - **Selección:** filter(Predicate)
   - **Transformación:** map(Function)
   - **Ordenación:** sorted(Comparator)
   - **Agregación/materialización:** collect(...), count(), toList()

4. **Diferencia esencial:**
   - **Imperativo:** “dime cómo lo haces” (control del proceso)
   - **Declarativo:** “dime qué quieres” (pipeline de operaciones)

Estilo Imperativo

```
List<Integer> resultado = new ArrayList();
  for (int x : nuneros) (
    if (x > 10) (
        resultado.add(x * 2);
    )
  )
```



Paso a paso, manual

Estilo Declarativo

```
List<Integer> resultado =
    nuneros.stream()
      .filter(x -> x > 10)
      .maplx ->> x * 2)
      .collect(Collectors.toList();
)
```

Qué se auiere, sin detalles

STREAMS

Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Recap: arrays vs colecciones (cuándo y por qué)

Arrays vs colecciones: criterio de elección.

En Java, tanto los arrays como las colecciones permiten almacenar conjuntos de datos en memoria, pero responden a necesidades distintas.

El punto clave no es “cuál es mejor”, sino **qué requisitos tiene el problema**: tamaño variable, operaciones frecuentes, acceso por clave, unicidad, orden, etc.

Este tema se centra en **colecciones** porque son la herramienta general para datos dinámicos y procesamiento moderno.

1. **Arrays: cuándo encajan bien**

   Un array es una estructura de tamaño fijo y acceso directo por índice. Es una buena opción cuando:

   - Conoces el número de elementos desde el inicio y no cambia (o cambia muy poco).
   - Necesitas rendimiento predecible y acceso rápido por índice.
   - Trabajas con datos “densos” (p. ej., tablas, matrices, buffers).

   Ejemplo típico:

   ```
   int[] notas = new int[30];   // tamaño fijo conocido
   notas[0] = 8;
   ```

2. **Limitaciones prácticas de arrays en programas reales**

   En cuanto el problema exige dinámica, aparecen fricciones:

   - Tamaño fijo: crecer implica crear otro array y copiar.
   - No existe una API rica para operaciones comunes (añadir/eliminar/filtrar/ordenar por criterios).
   - Modelar requisitos como “sin duplicados” o “búsqueda por clave” no es natural con arrays.

Gestión de Datos en Memoria

Introducción: datos en memoria y enfoque moderno

Recap: arrays vs colecciones (cuándo y por qué)

Arrays vs colecciones: criterio de elección.

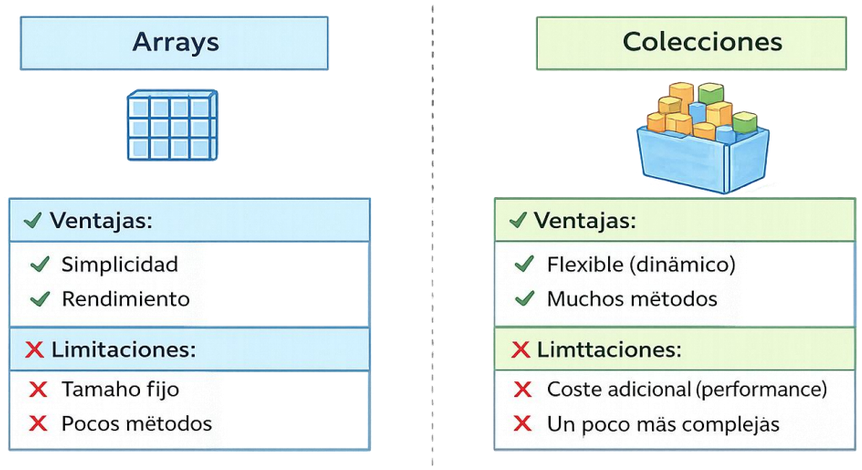
3. **Colecciones: por qué se usan en aplicaciones reales**

   Las colecciones del Java Collections Framework existen para cubrir necesidades típicas de software:

   - Tamaño variable (crecimiento/disminución durante ejecución).
   - Operaciones de alto nivel (añadir, eliminar, buscar, iterar, ordenar).
   - Estructuras con semántica específica:
     - **`List`:** secuencia ordenada, permite duplicados.
     - **`Set`:** unicidad (sin duplicados).
     - **`Map`:** acceso por clave (clave → valor).

   Ejemplo conceptual (sin entrar aún en implementaciones):

```
List<String> alumnos;          // secuencia
Set<String> dniUnicos;         // unicidad
Map<String, Double> notasPorDni; // clave → valor
```



4. **Regla de decisión rápida (lo que debéis interiorizar)**
   - ¿Necesito índice y orden de inserción? → `List`
   - ¿Necesito evitar duplicados? → `Set`
   - ¿Necesito buscar por identificador/clave? → `Map`
   - ¿Tamaño y/o acceso directo simple? → `array`

Los arrays son una herramienta básica y útil, pero las colecciones son el estándar para gestionar datos dinámicos y preparar el terreno para el procesamiento moderno (lambdas y streams) que veremos en los apartados posteriores.

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Arrays y Matrices: Declaración, Recorrido y Uso.

Arrays en java. Modelo, Acceso y Uso.

Los arrays son la forma más básica de representar un conjunto de datos en Java. Permiten almacenar múltiples elementos del mismo tipo en una estructura única, donde cada elemento se identifica por una posición (índice).

Definición y modelo de memoria

Un array se declara indicando el tipo de dato y su tamaño:

```
int[] numeros = new int[5];
```

Cuando se crea, Java reserva un bloque contiguo de memoria con espacio para 5 enteros. Cada posición del array se accede mediante un índice, comenzando en 0.

Esto implica que:

- La posición física de cada elemento está determinada desde el inicio
- El acceso a cualquier elemento es directo, sin necesidad de recorrer la estructura

Acceso y recorrido

El acceso a un elemento concreto es inmediato:

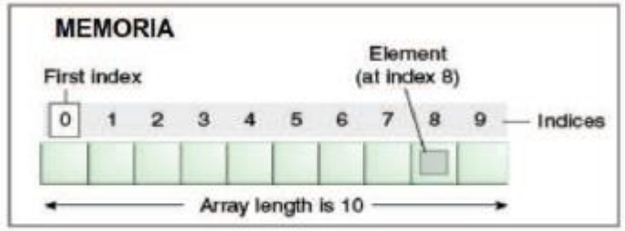
```
int valor = numeros[2];
```

No depende del tamaño del array → complejidad **O(1)**

Para procesar todos los elementos, se recorre mediante un bucle:

```
for (int i = 0; i < numeros.length; i++) {
    System.out.println(numeros[i]);
}
```

En este caso, el número de operaciones crece con el tamaño del array → **O(n)**



Tamaño del array: 10
Índices: del 0 al 9

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Arrays y Matrices: Declaración, Recorrido y Uso.

Arrays en java. Modelo, Acceso y Uso.

Inicialización y modificación

También es posible inicializar directamente:

```
int[] numeros = {10, 20, 30, 40, 50};
```

Y modificar valores en cualquier posición:

```
numeros[1] = 25;
```

Arrays multidimensionales

Los arrays pueden representar estructuras más complejas, como matrices:

```
int[][] matriz = new int[3][3];
matriz[0][1] = 5;
```

Aquí cada elemento se accede mediante múltiples índices, pero el principio sigue siendo el mismo: acceso directo por posición.

Implicación clave

El diseño de los arrays está optimizado para:

- Acceso rápido
- Estructuras simples
- Datos con tamaño conocido

Pero esta misma rigidez es lo que limita su uso en aplicaciones reales, donde los datos suelen crecer o cambiar dinámicamente.

Un array ofrece acceso eficiente y control directo sobre los datos, pero su falta de flexibilidad lo hace insuficiente para la mayoría de escenarios dinámicos.

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Análisis Básico de Complejidad: Origen y Cálculo.

Complejidad (Big-O): qué mide y cómo se calcula correctamente. $T(n) \in O(f(n)) \Leftrightarrow \exists c > 0, \exists n_0 \in N$ tal que $\forall n \geq n_0, T(n) \leq c \cdot f(n)$

La complejidad temporal es un modelo para estimar cómo crece el número de operaciones que ejecuta un algoritmo cuando crece el tamaño de la entrada, `n`. No mide "segundos"; **mide el orden de crecimiento dominante**. Formalmente, si `T(n)` es el número de pasos, decimos que un algoritmo es `O(f(n))` si existe una constante `c` y un `n0` tal que `T(n) ≤ c·f(n)` para todo `n ≥ n0`. En la práctica, esto se traduce en: nos interesa la forma de la curva de crecimiento, no el tiempo exacto.

1. **Qué es `n` y qué estamos contando**
   - `n` es el tamaño relevante del dato: longitud de un array, número de elementos de una lista, número de claves en un mapa, etc.
   - Contamos operaciones "elementales" repetidas: comparaciones, asignaciones, accesos, llamadas constantes. El objetivo no es exactitud absoluta, sino identificar el término que domina cuando `n` es grande.
2. **Método de cálculo (procedimiento)**
   a) Identifica el "bloque dominante": la parte que más se repite con `n`.
   b) Expresa el número de iteraciones en función de `n`.
   c) Multiplica por el coste interno (si es constante, se queda en constante)
   d) Quédate con el término dominante y elimina constantes (regla de Big-O).

Ejemplo lineal (recorrido completo)

```
for (int i = 0; i < a.length; i++) {   // a.length = n
    suma += a[i];                      // coste constante por iteración
}
```

- Iteraciones: **`n`**
- Coste por iteración: constante $\Rightarrow T(n) = k \cdot n + b \Rightarrow O(n)$

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Análisis Básicos de Complejidad: Origen y Cálculo.

Complejidad (Big-O): qué mide y cómo se calcula correctamente.

3. **Reglas prácticas de composición** (las que de verdad usas en análisis)
   - **Secuencia** (bloques uno detrás de otro): se suman y domina el mayor

     `O(n) + O(n) ⇒ O(n)`

     `O(n) + O(n log n) ⇒ O(n log n)`
   - **Bucles anidados**: se multiplican

     ```
     for (int i = 0; i < n; i++)
       for (int j = 0; j < n; j++)
         op();                    // constante
     ⇒ T(n) ≈ n·n ⇒ O(n²)
     ```
   - **Condicionales**: se toma el peor caso si no se especifica lo contrario
     - `if` puede ejecutar una rama u otra; si una es `O(n)` y otra `O(1)`, se reporta `O(n)`.
4. **Tres casos: mejor, promedio y peor (y por qué importa)**

   Hay algoritmos cuyo coste depende de los datos, no solo de `n`:
   - Búsqueda lineal en array:

     ```
     boolean contiene(int[] a, int x){
       for(int i=0;i<a.length;i++) if(a[i]==x) return true;
       return false;
     }
     ```
     - Mejor caso: `O(1)` (si está al principio)
     - Peor caso: `O(n)` (si está al final o no está)
     - Promedio: `~O(n)`

En este tema, cuando no se indique lo contrario, asumimos **peor caso**.

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Análisis Básicos de Complejidad: Origen y Cálculo.

Complejidad (Big-O): qué mide y cómo se calcula correctamente.

5. **Complejidad logarítmica** (por reducción a la mitad)

   Cuando en cada paso reduces el problema por un factor (p. ej., mitad), aparece `O(log n)`. Caso típico: búsqueda binaria (requiere datos ordenados). La intuición es: "¿cuántas veces puedo dividir `n` entre 2 hasta llegar a 1?".

6. **Amortizada** (porque aparecerá al hablar de estructuras dinámicas)

   A veces una operación es casi siempre barata, pero ocasionalmente cara (p. ej., redimensionar y copiar). El coste "amortizado" reparte el coste de esos picos sobre muchas operaciones. No es magia: es una media sobre secuencias largas de operaciones.

7. **Nota mínima sobre memoria** (complejidad espacial)

   Además del tiempo, a veces interesa cuánta memoria extra usa el algoritmo: `O(1)` **extra (in-place)** frente a `O(n)` **extra** (copias, estructuras auxiliares). En colecciones, esto será relevante al comparar estructuras y operaciones que materializan resultados.

   El objetivo de Big-O en este tema no es "matemáticas": es poder justificar decisiones.

Si entiendes de dónde sale `O(n)`, `O(log n)` o `O(n log n)`, entonces puedes razonar por qué una estructura/operación es apropiada o no para un caso de uso.

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Análisis Básicos de Complejidad: Origen y Cálculo.

Complejidad (Big-O): De dónde salen O(1), O(n), O(log n) y O(n log n)

Vamos a identificar patrones típicos de código y asociarles su crecimiento.

1. **O(1): coste constante (no depende de n)** Una operación es O(1) cuando ejecuta un número fijo de pasos, independientemente del tamaño del conjunto de datos.

   Ejemplos:

```
int x = a[5];          // acceso por índice (si existe)
int y = 7 * 3 + 2;     // cálculo fijo
```

- No hay bucles ni repetición proporcional a n. ⇒ El número de pasos es constante ⇒ **O(1).**

2. **O(n): coste lineal (recorrer n elementos una vez)** Aparece cuando ejecutas un bucle que itera proporcionalmente al tamaño de la entrada.

   Ejemplo:

```
for (int i = 0; i < n; i++) {
    op();              // op es constante
}
```

- `op()` se ejecuta exactamente n veces. ⇒ Crecimiento lineal ⇒ **O(n).**

3. **O(log n): coste logarítmico (reducir el problema por factor constante)** Aparece cuando en cada paso reduces el tamaño del problema multiplicativamente (por ejemplo, a la mitad). La pregunta que responde `log n` es: "¿cuántas veces puedo dividir n entre 2 hasta llegar a 1?"

   Ejemplo típico (idea de búsqueda binaria / reducción):

   - Iteraciones: n → n/2 → n/4 → … → 1
   - Número de pasos ≈ log2(n) ⇒ **O(log n).**

```
int m = n;
while (m > 1) {
    m = m / 2;
    op();          // constante
}
```

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Análisis Básicos de Complejidad: Origen y Cálculo.

Complejidad (Big-O): De dónde salen O(1), O(n), O(log n) y O(n log n)

4. **O(n log n): combinar un recorrido lineal con una fase logarítmica** Aparece cuando haces una operación logarítmica "por cada elemento" o cuando usas algoritmos de ordenación basados en dividir y combinar.

   Dos formas típicas de verlo:

   A) "log n por cada elemento":

```
for (int i = 0; i < n; i++) {
    opLogaritmica();   // cuesta O(log n)
}
```

   - Se repite n veces algo que cuesta log n.
     ⇒ n · log n ⇒ O(n log n)

   B) Ordenación por divide-and-conquer (intuición del sort):

   - Dividir repetidamente: log n niveles de división
   - En cada nivel se procesan n elementos en total (combinar/mezclar)
     ⇒ n por nivel × log n niveles ⇒ O(n log n)

Resumen visual (patrones → Big-O)

- **O(1)**: pasos fijos, sin bucles dependientes de n
- **O(n):** un bucle proporcional a n (una pasada)
- **O(log n):** bucle que reduce el problema por mitad / factor constante
- **O(n log n):** n elementos × log n niveles/pasos (ordenación o "log por elemento")

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Limitaciones de arrays.

Impacto en el diseño y la complejidad.

Los arrays proporcionan acceso rápido y estructura simple, pero presentan limitaciones importantes cuando los datos son dinámicos o requieren operaciones complejas. Estas limitaciones no son teóricas: derivan directamente de cómo están implementados en memoria.

1) Tamaño fijo: falta de adaptabilidad

```
int[] datos = new int[5];
```

El tamaño se define en el momento de la creación y no puede cambiar.

Si necesitamos más espacio:

```
int[] nuevo = new int[10];
System.arraycopy(datos, 0, nuevo, 0, datos.length);
```

- Se crea un nuevo array y se copian los elementos
- Coste proporcional al número de elementos → **O(n)**

2) Inserciones y eliminaciones costosas

Insertar en una posición intermedia implica desplazar elementos:

```
// Insertar en posición 0
for (int i=(n-1); i>0; i--) {
    datos[i] = datos[i - 1];
}
datos[0] = nuevoValor;
```

Se desplazan n elementos → **O(n)**

Eliminar ocurre lo mismo:

```
for (int i=pos; i<(n-1); i++) {
    datos[i] = datos[i+1];
}
```

De nuevo desplazamiento → **O(n)**

Gestión de Datos en Memoria

Arrays y Utilidades Básicas

Limitaciones de arrays.

Impacto en el diseño y la complejidad.

3) Búsqueda no optimizada

Si no hay orden ni estructura adicional:

```
for (int i = 0; i < n; i++) {
    if (datos[i] == valor) return i;
}
```

En el peor caso recorre todo → **O(n)**

4) Falta de semántica estructural

Un array no expresa intención:

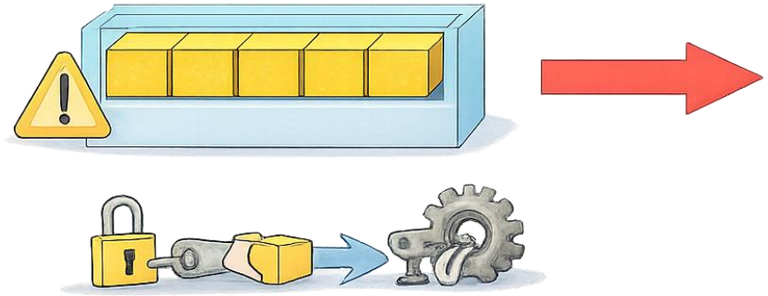
- No indica si hay duplicados
- No distingue claves de valores
- No define orden lógico más allá del índice

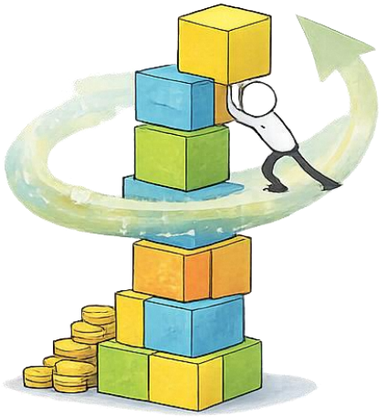
Todo debe gestionarse manualmente en el código

5) Consecuencia técnica

Las limitaciones de arrays aparecen cuando:

- El número de datos es variable
- Se requieren inserciones/eliminaciones frecuentes
- Se necesitan búsquedas eficientes
- Se quiere expresar lógica sobre conjuntos de datos





Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Concepto de Colección.

Qué es una colección en java.

En Java, una colección es una abstracción que representa un conjunto de objetos sobre los que se pueden realizar operaciones de forma uniforme. A diferencia de los arrays, no define únicamente "cómo almacenar", sino también "qué operaciones están disponibles" sobre ese conjunto.

Definición técnica

Una colección es un objeto que:

- Agrupa múltiples elementos del mismo tipo lógico
- Proporciona una interfaz común de operaciones
- Oculta los detalles de implementación interna

Forma parte del **Java Collections Framework (JCF)**

Modelo conceptual

Una colección introduce dos niveles diferenciados:

- **Interfaz** → define el comportamiento (qué se puede hacer)
- **Implementación** → define la estructura interna (cómo se hace)

Ejemplo típico:

```
List<String> nombres = new ArrayList<>();
```

- `List` → contrato (operaciones disponibles)
- `ArrayList` → implementación concreta

Operaciones comunes en colecciones

Independientemente de la implementación, las colecciones permiten:

- Añadir elementos
- Eliminar elementos
- Recorrer el conjunto
- Consultar contenido
- Obtener tamaño

API homogénea → mismo uso, distintas estructuras

Implicación técnica

Las colecciones permiten:

- Adaptarse a datos dinámicos
- Reducir código repetitivo
- Elegir la estructura según el problema

Separan el **uso** de la **implementación**

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de Framework de Colecciones.

Estructura del Java Collections Framework (JCF).

El Java Collections Framework (JCF) organiza las estructuras de datos en una jerarquía de interfaces y clases que separa claramente **el comportamiento** (qué se puede hacer) de **la implementación** (cómo se hace internamente).

1) Idea estructural del framework

El JCF se basa en una jerarquía de interfaces:

- `Collection` → raíz de estructuras lineales
- `List` → secuencias ordenadas
- `Set` → conjuntos sin duplicados
- `Map` → estructuras clave → valor (no hereda de Collection)

Esta jerarquía permite tratar distintas estructuras de forma uniforme

2) Representación simplificada

```
Collection
  ├── List
  ├── Set
Map   (jerarquía independiente)
```

3) Separación clave: interfaz vs implementación

Ejemplo:

```
List<String> lista = new ArrayList<>();
```

- `List` → define operaciones (add, get, remove…)
- `ArrayList` → define cómo se almacenan los datos

El código depende de la interfaz, no de la implementación

4) Consecuencia técnica

Gracias a esta jerarquía:

- Podemos cambiar la implementación sin cambiar el código cliente
- Podemos programar contra abstracciones
- Se reduce el acoplamiento

Ejemplo:

```
List<String> lista = new LinkedList<>();
```

Mismo código, distinta estructura interna

5) Diferencia fundamental dentro del modelo

- `List` → orden + duplicados
- `Set` → sin duplicados
- `Map` → acceso por clave

No son variantes, son **modelos de datos distintos**

El JCF no es un conjunto de clases aisladas,
sino una jerarquía coherente que separa comportamiento e implementación.

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de Framework de Colecciones.

Jerarquía Completa con Interfaces e Implementaciones.

El Java Collections Framework no solo define interfaces, sino también múltiples implementaciones concretas para cada tipo de colección. Cada implementación resuelve el mismo problema con distintas características (rendimiento, orden, estructura interna).

1) Jerarquía ampliada (visión real)

```
Collection
  ├── List
  │     ├── ArrayList
  │     └── LinkedList
  │
  └── Set
        ├── HashSet
        ├── LinkedHashSet
        └── TreeSet
Map
  ├── HashMap
  ├── LinkedHashMap
  └── TreeMap
```

2) Interpretación de la jerarquía

- **Interfaces (nivel abstracto):**
  - `List`, `Set`, `Map`
  - Definen qué operaciones existen
- **Clases (nivel concreto):**
  - `ArrayList`, `HashSet`, `HashMap`, etc.
  - Definen cómo se implementan esas operaciones

Una misma interfaz puede tener múltiples implementaciones

3) Diferencias clave entre implementaciones

Aunque comparten interfaz, cambian propiedades fundamentales:

- `ArrayList` vs `LinkedList` → estructura interna distinta
- `HashSet` vs `TreeSet` → orden vs rendimiento
- `HashMap` vs `TreeMap` → acceso rápido vs ordenado

La elección impacta directamente en el comportamiento y la complejidad

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de Framework de Colecciones.

Jerarquía Completa con Interfaces e Implementaciones.

4) Ejemplo de uso real

```
Set<String> nombres = new HashSet<>();
```

Podemos cambiar la implementación:

```
Set<String> nombres = new TreeSet<>();
```

El código sigue funcionando, pero cambia:

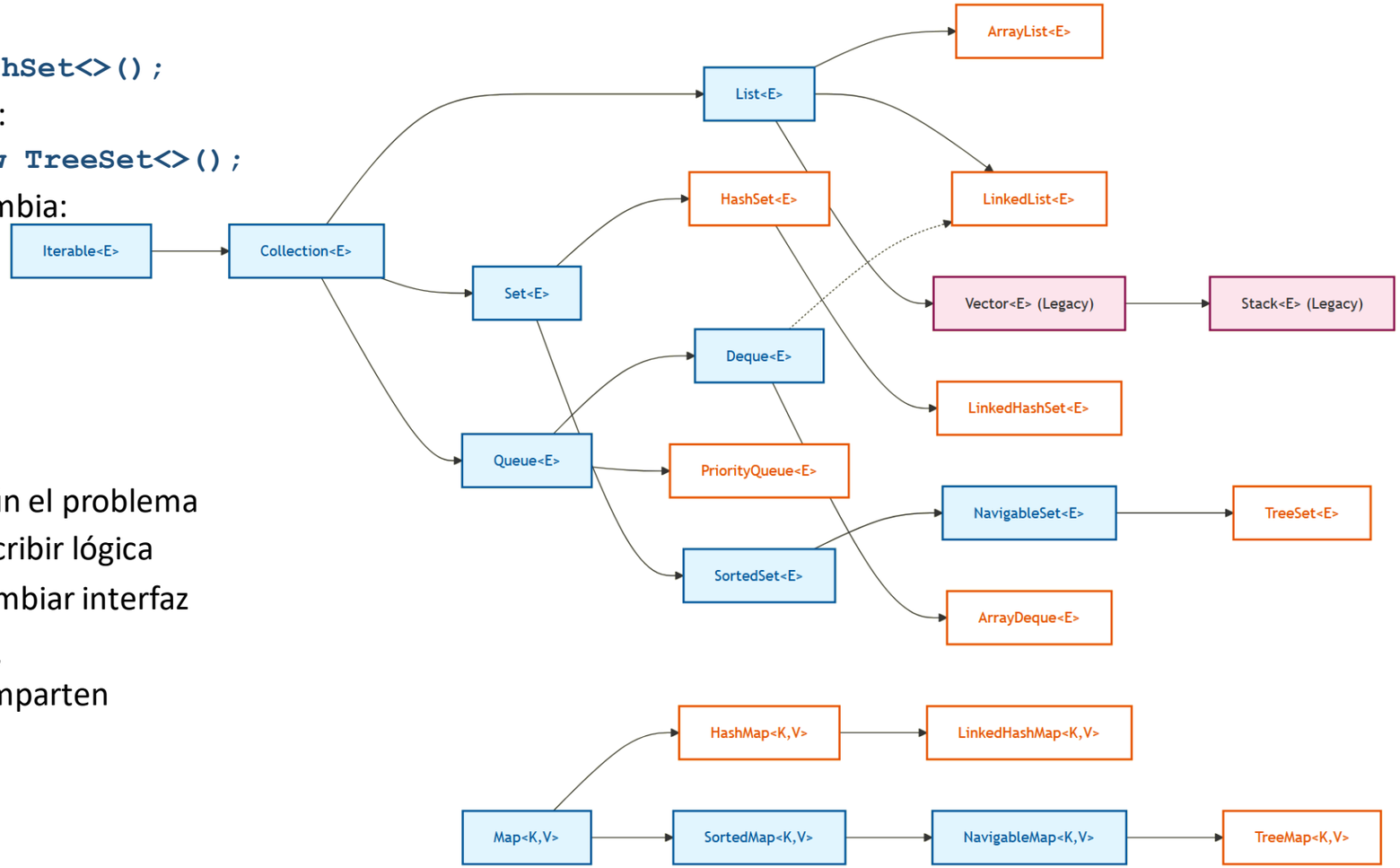
- el orden
- el rendimiento
- el comportamiento interno

5) Implicación técnica importante

El diseño del JCF permite:

- Elegir la estructura adecuada según el problema
- Cambiar implementación sin reescribir lógica
- Optimizar comportamiento sin cambiar interfaz

En Java no trabajamos con “una colección”,
sino con una familia de estructuras que comparten
interfaz pero difieren en comportamiento.



Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Generics y Tipado Seguro.

Colecciones Tipadas y Seguridad en Tiempos de Compilación.

Las colecciones en Java almacenan objetos. Sin tipado genérico, una colección podría mezclar tipos y obligar a usar conversiones (casts) al recuperar elementos, lo que desplaza errores al tiempo de ejecución. Los **genéricos (Generics)** resuelven esto permitiendo declarar el tipo de elementos que una colección puede contener.

1. **Problema sin genéricos: pérdida de seguridad de tipos**

   En Java "pre-genéricos", una colección almacenaba `Object`:

   ```
   List lista = new ArrayList();
   lista.add("Ana");
   lista.add(10);                  // mezcla de tipos
   String s = (String) lista.get(1);  // ClassCastException en ejecución
   ```

   Aquí el compilador no puede garantizar que `get(1)` devuelva un `String`. El error se detecta tarde (en ejecución), y además obliga a introducir casts, que añaden ruido y riesgo.

2. **Solución: declarar el tipo de elementos con `<T>`**

   Con genéricos, el tipo se especifica explícitamente y el compilador valida todas las inserciones y extracciones:

   ```
   List<String> lista = new ArrayList<>();
   lista.add("Ana");
   // lista.add(10);               // ERROR de compilación
   String s = lista.get(0);        // sin cast
   ```

   Esto aporta dos beneficios directos:

   - **Type safety**: evita mezclas incorrectas de tipos.
   - **Legibilidad**: el tipo esperado está documentado en la propia declaración.

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Generics y Tipado Seguro.

Colecciones Tipadas y Seguridad en Tiempos de Compilación.

3.Parámetros genéricos típicos en el framework

- `List<T>` / `Set<T>`: un tipo de elemento
- `Map<K, V>`: tipo de clave y tipo de valor

Ejemplos:

```
Set<Integer> edades = new HashSet<>();
Map<String, Double> notas = new HashMap<>();
```

4.Relación con primitivos: autoboxing y wrappers

Las colecciones no aceptan primitivos (`int`, `double`), sino objetos (`Integer`, `Double`). Java convierte automáticamente mediante autoboxing/unboxing:

```
List<Integer> xs = new ArrayList<>();
xs.add(5);            // autoboxing: int -> Integer
int a = xs.get(0);    // unboxing: Integer -> int
```

Los genéricos permiten que los errores de tipo se detecten en compilación, eliminan casts y hacen que el uso de colecciones sea robusto y mantenible.

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de FrameWork de Colecciones.

Por qué Map no es Collection y Cómo se Recorre Correctamente.

En el Java Collections Framework, `Map` no forma parte de `Collection`.
Esta distinción no es accidental: responde a un modelo de datos diferente.

1) Diferencia estructural clave

- `Collection` (List, Set) → colección de **elementos individuales**
- `Map` → colección de **pares clave → valor**

  En `Map` cada elemento está compuesto por dos partes:

  - clave (única)
  - valor (asociado)

  Por eso **no encaja en el modelo de Collection**

2) Consecuencia en el diseño

`Map` define su propia jerarquía y su propia API:

- `put(K, V)` → inserta pares
- `get(K)` → acceso por clave
- `containsKey(K)` → búsqueda por clave

No existen operaciones típicas de `Collection` como `add()`

3) Cómo se recorre un Map correctamente

Un `Map` no se recorre como una lista. Hay tres formas principales:

A) Recorrer claves (keySet)

```
for (String clave : map.keySet()) {
    System.out.println(clave);
}
```

Solo accedes a las claves

B) Recorrer valores (values)

```
for (Integer valor : map.values()) {
    System.out.println(valor);
}
```

Solo accedes a los valores

C) Recorrer pares clave-valor (forma recomendada)

```
for (Map.Entry<String, Integer> e : map.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

Acceso completo y eficiente

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de FrameWork de Colecciones.

Por qué Map no es Collection y Cómo se Recorre Correctamente.

4) Interpretación técnica

- `entrySet()` devuelve un conjunto de objetos `Entry<K,V>`
- Cada `Entry` representa un par clave → valor
- Evita búsquedas adicionales (más eficiente que keySet + get)

5) Error típico

```
for (String k : map.keySet()) {
    System.out.println(map.get(k));
}
```

Funciona, pero:

- Hace una búsqueda adicional por cada clave
- Menos eficiente

`Map` no es una colección de elementos, sino una estructura de asociaciones clave → valor con su propia lógica de acceso y recorrido.

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de Framework de Colecciones.

Elección de estructura (List vs Set vs Map).

Elegir correctamente entre `List`, `Set` y `Map` no es una cuestión de preferencia; depende de la semántica del dato y de las operaciones dominantes (cómo se insertan, cómo se buscan, si hay duplicados, si se necesita una clave, etc.). Esta decisión condiciona comportamiento, complejidad y claridad del código.

1) `List` — Secuencia ordenada (permite duplicados)

Usa `List` cuando el dato se interpreta como una **secuencia**:

- El **orden importa** (inserción o posición)
- Puede haber **duplicados**
- Acceso frecuente por **índice** o recorrido secuencial

Ejemplos típicos:

- historial de acciones (orden temporal)
- lista de alumnos en orden de llegada
- carrito de compra (puede repetirse un producto)

Modelo mental: “una fila de elementos”.

2) `Set` — Conjunto (no permite duplicados)

Usa `Set` cuando lo relevante es la **unicidad**:

- No quieres elementos repetidos
- Te interesa comprobar pertenencia (“¿está X?”)
- El orden puede ser irrelevante o depender de la implementación

Ejemplos típicos:

- DNI/IDs únicos
- permisos/roles sin repetición
- etiquetas únicas asociadas a un objeto

Modelo mental: “un conjunto matemático”.

Gestión de Datos en Memoria

Framework de Colecciones y Genéricos

Jerarquía de Framework de Colecciones.

Elección de estructura (List vs Set vs Map).

3) `Map` — Asociación clave → valor (búsqueda por clave)

Usa `Map` cuando necesitas acceder a datos mediante una **clave**:

- Cada elemento se identifica por un **id** o clave única
- Necesitas obtener el valor asociado de forma eficiente
- La operación dominante es "buscar por clave"

Ejemplos típicos:

- `dni → nota`
- `idPedido → Pedido`
- `usuario → n° de compras` (acumulación)

Modelo mental: "diccionario / índice".

4) Preguntas de decisión (regla operativa)

Para elegir estructura, responde en este orden:

1. ¿Necesito buscar por una clave identificadora? → **Sí**: `Map<K,V>`
2. Si no hay clave: ¿debo evitar duplicados? → **Sí**: `Set<T>`
3. Si permite duplicados o importa el orden: → `List<T>`

5) Ejemplos directos (mapeo problema → estructura)

- "Usuarios conectados en orden de conexión" → `List<Usuario>`
- "DNI de alumnos (sin repetidos)" → `Set<String>`
- "Nota por DNI" → `Map<String, Double>`
- "Contar apariciones de palabras" → `Map<String, Integer>`

Idea clave

`List` modela secuencias, `Set` modela unicidad y `Map` modela acceso por clave.

La estructura correcta es la que hace que el problema sea "natural" de expresar en código.

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

List: Modelo de Datos y Operaciones Fundamentales.

Tras entender la jerarquía del framework, comenzamos con la primera estructura concreta: `List`.
Una `List` representa una **secuencia ordenada de elementos**, donde cada elemento ocupa una posición y puede aparecer más de una vez.

1) Modelo conceptual de List

Una `List` se comporta como una secuencia:

- Cada elemento tiene un **índice** (posición)
- El orden se **mantiene**
- Se permiten **duplicados**

Ejemplo:

```
List<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Ana");
```

Resultado: `[Ana, Luis, Ana]`

2) Operaciones fundamentales

Las operaciones que definen el uso de una `List` son:

- **Inserción:** `nombres.add("Marta");`
- **Acceso por índice:** `String n = nombres.get(1);`
- **Modificación:** `nombres.set(0, "Carlos");`
- **Eliminación:** `nombres.remove(1);`
- **Tamaño:** `int total = nombres.size();`

3) Interpretación técnica

Una `List` está diseñada para:

- Acceder a elementos por posición
- Recorrer secuencias de datos
- Mantener orden de inserción

Es la estructura más cercana a un array, pero con mayor flexibilidad

4) Coste de operaciones (visión general)

Aunque depende de la implementación (`ArrayList` o `LinkedList`):

- Acceso por índice → normalmente **O(1)** o cercano
- Inserción al final → eficiente
- Inserciones en medio → pueden implicar coste lineal

La implementación concreta determina el rendimiento

5) Cuándo usar List

Usa `List` cuando:

- El orden de los elementos sea relevante
- Se permitan duplicados
- Se trabaje con datos secuenciales

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

Patrones de uso de List: Creación, Carga y Recorrido.

Una `List` no solo define un modelo de datos, sino también múltiples formas de creación, inicialización y recorrido. Conocer estos patrones permite escribir código más eficiente, expresivo y adecuado al contexto.

1) Creación y carga incremental (forma estándar)

```
List<String> nombres = new ArrayList<>();

nombres.add("Ana");
nombres.add("Luis");
nombres.add("Marta");
```

Uso típico cuando los datos se generan dinámicamente
Permite control total sobre inserciones

2) Inicialización directa (datos conocidos)

```
List<String> nombres = List.of("Ana","Luis","Marta");
```

- Lista **inmutable**
- No permite `add()` ni `remove()`

Útil para datos constantes o de prueba

3) Conversión desde array

```
String[] datos = {"Ana", "Luis", "Marta"};
List<String> lista = Arrays.asList(datos);
```

- Tamaño fijo (no se puede añadir/eliminar)
- Permite modificar elementos existentes

Caso intermedio entre array y lista dinámica

4) Conversión a lista modificable

```
List<String> lista = new ArrayList<>(Arrays.asList(datos));
```

Permite añadir y eliminar elementos
Muy usado en práctica real

5) Recorrido de la lista

A) Bucle clásico

```
for (int i = 0; i < lista.size(); i++) {
    System.out.println(lista.get(i));
}
```

Control por índice

B) For-each (más habitual)

```
for (String s : lista) {
    System.out.println(s);
}
```

Más limpio y legible

C) forEach (puente hacia programación funcional)

```
lista.forEach(System.out::println);
```

Introduce estilo declarativo

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

Patrones de uso de List: Creación, Carga y Recorrido.

6) Interpretación técnica

- Existen múltiples formas de construir una `List` según el origen de los datos.
- No todas las listas son iguales:
  - algunas son **inmutables**
  - otras tienen **tamaño fijo**
  - otras son completamente dinámicas

Elegir la forma de creación afecta directamente al comportamiento

No todas las listas se crean ni se usan igual:
el origen de los datos y la necesidad de modificación determinan el patrón adecuado.

ArrayList: Modelo Interno, redimensionado y Complejidad.

Complejidad de operaciones del ArrayList:

Operación	Coste	Justificación
`get(i)`	O(1)	acceso directo al array
`add(e)`	O(1) amortizado	normalmente añade al final
`add(i,e)`	O(n)	desplazamiento de elementos
`remove(i)`	O(n)	desplazamiento

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

ArrayList: Modelo Interno, redimensionado y Complejidad.

`ArrayList` es la implementación más utilizada de `List`. Su comportamiento se entiende correctamente si se analiza cómo está construido internamente: un **array dinámico** que gestiona su propia capacidad.

1) Modelo interno: array dinámico

Un `ArrayList` mantiene internamente un array:

```
Object[] elementos;
```

- Los elementos se almacenan de forma **contigua**
- Existe una diferencia entre:
  - **tamaño (size)** → elementos reales
  - **capacidad (capacity)** → tamaño del array interno

La capacidad suele ser mayor que el tamaño para permitir crecimiento

2) Redimensionado (crecimiento automático)

Cuando el array interno se llena:

1. Se crea un nuevo array más grande
2. Se copian los elementos
3. Se añade el nuevo elemento

```
// operación conceptual
nuevoArray = copiar(arrayAnterior,nuevaCapacidad);
```

Este proceso tiene coste **O(n)**

4) ¿Por qué "amortizado"?

Aunque insertar puede costar **O(n)** al redimensionar:

- ocurre **pocas veces**
- la mayoría de inserciones son **O(1)**

Promedio a largo plazo → **O(1) amortizado**

5) Implicaciones prácticas

`ArrayList` es eficiente cuando:

- Se accede frecuentemente por índice
- Se añaden elementos al final
- El número de inserciones intermedias es bajo

6) Limitaciones derivadas del diseño

Debido a su base en array:

- Inserciones en medio → costosas
- Eliminaciones → costosas
- Redimensionado → coste puntual elevado

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

LinkedList: Modelo Interno, Operaciones y Comparación con ArrayList.

`LinkedList` es una implementación de **`List`** basada en una **estructura enlazada**, no en un array. Esto cambia completamente el comportamiento de las operaciones respecto a **`ArrayList`**.

1) Modelo interno: lista doblemente enlazada

Cada elemento (nodo) contiene:

- el dato
- referencia al siguiente nodo
- referencia al nodo anterior

Representación conceptual:

```
[prev | dato | next] <-> [prev | dato | next] <-> ...
```

Los elementos **no están en memoria contigua**

El acceso no es directo, sino secuencial

2) Consecuencia directa del modelo

Para acceder a un elemento en posición `i`:

```
lista.get(i);
```

- Hay que recorrer la lista nodo a nodo
  Complejidad: **O(n)**

3) Inserciones y eliminaciones

Insertar o eliminar en posiciones conocidas:

```
lista.add(0, "Ana");
lista.remove(0);
```

- No hay desplazamiento de elementos
- Solo se actualizan enlaces

Complejidad: **O(1)** (si ya estás en el nodo)

4 y 5) Complejidad: Comparación con ArrayList

Operación	ArrayList	LinkedList
Acceso (`get`)	O(1)	O(n)
Inserción final	O(1) amortizado	O(1)
Inserción en medio	O(n)	O(n)
Eliminación	O(n)	O(n)

6) Interpretación técnica real

Aunque **`LinkedList`** evita desplazamientos:

- necesita recorrer la lista para encontrar la posición
- esto anula muchas de sus ventajas en la práctica

Por eso se usa menos que **`ArrayList`**

7) Cuándo usar LinkedList

Tiene sentido cuando:

- Hay muchas inserciones/eliminaciones en extremos
- Se usa como cola o deque
- No se necesita acceso frecuente por índice

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

Iterator y Modificación Estructural.

1) Problema típico

Cuando se recorre una colección con *for-each* no se puede modificar estructuralmente la colección durante el recorrido:

```
List<String> lista = new ArrayList<>();
lista.add("Ana");
lista.add("Luis");
lista.add("Marta");
for (String s : lista) {
    if (s.equals("Luis")) {
        lista.remove(s);   //
ConcurrentModificationException
    }
}
```

¿Por qué ocurre?

El *for-each* utiliza internamente un `Iterator`.
Si la colección cambia estructuralmente fuera del iterador, el sistema detecta la modificación y lanza excepción.

2) Qué es modificación estructural

Una modificación estructural es:

- add()
- remove()
- clear()
- cualquier operación que cambie el tamaño interno

No lo es:

- set(i, valor) en ArrayList
- modificar atributos internos de un objeto almacenado

3) Forma correcta: usar Iterator explícito

```
Iterator<String> it = lista.iterator();

while (it.hasNext()) {
    String s = it.next();
    if (s.equals("Luis")) {
        it.remove();   // Eliminación segura
    }
}
```

Aquí:

- El iterador controla el recorrido
- La eliminación se hace a través del propio iterador
- No se rompe la coherencia interna

4) Interpretación técnica

Las colecciones del JCF son *fail-fast*:

- Detectan modificaciones estructurales externas
- Lanzan ConcurrentModificationException
- Evitan estados inconsistentes

Esto no es concurrencia de hilos.
Es modificación durante iteración.

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

List: ArrayList y LinkedList.

Ejemplos Completos con List:

ArrayList: Gestion de una Lista de Pedidos.

```
List<String> pedidos = new ArrayList<>();
// Añadir elementos
pedidos.add("Pedido1");
pedidos.add("Pedido2");
pedidos.add("Pedido3");
// Recorrer
for (String p : pedidos) {
    System.out.println(p);
}
// Modificar
pedidos.set(1, "PedidoModificado");
// Eliminar
pedidos.remove(0);
// Resultado final
System.out.println(pedidos);
```

Qué se está trabajando

- Secuencia ordenada
- Modificación por índice
- Eliminación
- Recorrido

LinkedList como cola (FIFO)

```
LinkedList<String> cola = new LinkedList<>();
// Añadir elementos al final (enqueue)
cola.addLast("Pedido1");
cola.addLast("Pedido2");
cola.addLast("Pedido3");
// Procesar elementos (dequeue)
while (!cola.isEmpty()) {
    String pedido = cola.removeFirst();
    System.out.println("Procesando: " + pedido);
}
```

Qué se está trabajando

- Inserciones en el extremo → `addLast()`
- Eliminaciones en el extremo → `removeFirst()`
- Procesamiento secuencial (FIFO)

Modelo de **cola**

Por qué aquí LinkedList tiene sentido

- No hay accesos por índice
- No hay desplazamientos de elementos
- Operaciones en extremos → **O(1)**

Comparación implícita con ArrayList

Si esto se hiciera con `ArrayList`:

- `remove(0)` → **O(n)**
- `add(0, e)` → **O(n)**

En `LinkedList` → O(1)

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Set: Modelo de Datos y Propiedades Fundamentales.

Tras `List`, introducimos `Set`, que modela un concepto distinto: un **conjunto sin elementos duplicados**. Aquí la prioridad no es el orden ni la posición, sino la **unicidad**.

1) Modelo conceptual de Set

Un `Set` representa:

- Un conjunto de elementos **sin duplicados**
- Sin acceso por índice
- Con operaciones centradas en pertenencia

Ejemplo:

```
Set<String> nombres = new HashSet<>();
nombres.add("Ana");
nombres.add("Luis");
nombres.add("Ana");
```

Resultado: `["Ana", "Luis"]` (el duplicado no se añade)

2) Propiedad clave: unicidad

La operación `add()`:

- Inserta el elemento si no existe
- Ignora el elemento si ya está presente

```
boolean añadido = nombres.add("Ana");
```

Devuelve:

- `true` si se inserta
- `false` si ya existía

3) Operaciones fundamentales

- **Inserción:** `set.add("dato");`
- **Comprobación:** `set.contains("dato");`
- **Eliminación:** `set.remove("dato");`
- **Recorrido:**

```
for (String s : set) {
    System.out.println(s);
}
```

4) Implicación técnica

Un `Set` está optimizado para:

- Evitar duplicados automáticamente
- Comprobar pertenencia de forma eficiente
- Gestionar conjuntos de elementos únicos

No está pensado para acceso por posición

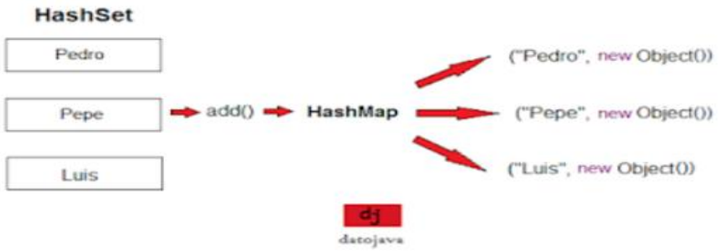
5) Diferencia fundamental con List

Característica	List	Set
Orden	Sí	No garantizado
Duplicados	Sí	No
Índice	Sí	No

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.



HashSet: Modelo Inerno, hashCode/equals y Complejidad.

`HashSet` es la implementación más usada de `Set`. Su objetivo es garantizar **unicidad** y **operaciones rápidas de inserción y búsqueda**. Para ello utiliza internamente una estructura basada en hashing.

1) Modelo interno: tabla hash (idea operativa)

Un `HashSet` se apoya en una tabla (array de "cubetas/buckets"):

1. A cada elemento se le calcula un **hash** (`hashCode()`)
2. Ese hash se transforma en un **índice** de la tabla
3. El elemento se almacena en esa posición (o en su bucket)

Representación simplificada:

```
índice:   0     1     2     3     4
         [ ]   [A]   [ ]   [B]   [C, D]
```

Si varios elementos caen en el mismo índice → **colisión**

2) Inserción y comprobación de duplicados

Al hacer `add(e)`:

- Se calcula `hashCode()` → determina el bucket
- Dentro del bucket se compara con `equals()`
- Si existe un elemento "igual" → **no se inserta**

```
set.add(e);
```

Unicidad = combinación de **hashCode + equals**

3) Contrato fundamental (imprescindible)

Para que `HashSet` funcione correctamente:

Si dos objetos son iguales (`equals`), deben tener el mismo `hashCode`

Si no se cumple:

- Se pueden insertar "duplicados lógicos"
- El comportamiento del conjunto será incorrecto

4) Complejidad de operaciones

Operación	Coste medio	Justificación
`add(e)`	O(1)	acceso directo por hash
`contains(e)`	O(1)	búsqueda por hash
`remove(e)`	O(1)	acceso directo

En caso de muchas colisiones:

- Puede degradar a **O(n)**

5) Propiedades clave de HashSet

- No mantiene orden
- No permite duplicados
- Alta eficiencia en inserción y búsqueda

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

HashSet: Modelo Inerno, hashCode/equals y Complejidad.

```java
import java.util.ArrayList;
import java.util.HashSet;
import java.util.Iterator;
import java.util.List;
import java.util.Set;
public class HashSet1 {
    public static void main(String[] args) {
        Set<String> frutas = new HashSet<>();
        frutas.add("Manzana");
        frutas.add("Platano");
        System.out.println("frutas está vacía = "+frutas.isEmpty());
        System.out.println("frutas contiene Manzana = "+frutas.contains("Manzana"));
        System.out.println("frutas contiene Mango = "+frutas.contains("Mango"));
        System.out.println("Manzana borrada del set frutas = "+frutas.remove("Manzana"));
        System.out.println("Mango borrado del set frutas = "+frutas.remove("Mango"));
        System.out.println("tamaño del set frutas = "+frutas.size());
        List<String> list = new ArrayList<>();
        list.add("Manzana");
        list.add("Platano");
        list.add("Mango");
        System.out.println("set frutas antes de añadir = "+frutas);
        System.out.println("list = "+list);
        frutas.addAll(list);
        System.out.println("set frutas después = "+frutas);
        Iterator<String> iterator = frutas.iterator();
        while(iterator.hasNext()){
            System.out.println("frutas consumidas "+iterator.next());
        }
         frutas.add("Naranja");
         System.out.println("set frutas antes de borrar = "+frutas);
         System.out.println("list = "+list);
         frutas.removeAll(list);
         System.out.println("set frutas despues de borrar = "+frutas);
         frutas.clear();
         System.out.println("set frutas está vacío = "+frutas.isEmpty());
    }
}
```

```
run:
frutas está vacía = false
frutas contiene Manzana = true
frutas contiene Mango = false
Manzana borrada del set frutas = true
Mango borrado del set frutas = false
tamaño del set frutas = 1
set frutas antes de añadir = [Platano]
list = [Manzana, Platano, Mango]
set frutas después = [Platano, Manzana, Mango]
frutas consumidas Platano
frutas consumidas Manzana
frutas consumidas Mango
set frutas antes de borrar = [Naranja, Platano, Manzana, Mango]
list = [Manzana, Platano, Mango]
set frutas despues de borrar = [Naranja]
set frutas está vacío = true
BUILD SUCCESSFUL (total time: 0 seconds)
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

LinkedHashSet y TreeSet: orden, estructura y coste.

Tras **`HashSet`**, existen otras implementaciones de **`Set`** que modifican dos aspectos clave: **el orden** y la **estructura interna**. Esto afecta directamente al comportamiento y a la complejidad de las operaciones.

1) LinkedHashSet — mantiene el orden de inserción

`LinkedHashSet` combina:

- Tabla hash (como **`HashSet`**)
- Lista enlazada interna para mantener el orden

Ejemplo:

```
Set<String> nombres = new LinkedHashSet<>();

nombres.add("Luis");
nombres.add("Ana");
nombres.add("Marta");
```

Resultado: `[Luis, Ana, Marta]` (orden preservado)

Características técnicas

- Mantiene el **orden de inserción**
- Permite iterar en ese orden
- Coste ligeramente mayor que **`HashSet`**

Complejidad

Operación	Coste
`add`	O(1)
`contains`	O(1)

Similar a `HashSet`, con pequeño overhead

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

LinkedHashSet y TreeSet: orden, estructura y coste.

2) TreeSet — conjunto ordenado automáticamente

`TreeSet` se basa en una estructura de árbol (normalmente un árbol balanceado):

- Los elementos se almacenan **ordenados automáticamente**
- No mantiene orden de inserción
- No usa hashing

Ejemplo:

```
Set<String> nombres = new TreeSet<>();

nombres.add("Luis");
nombres.add("Ana");
nombres.add("Marta");
```

Resultado: `[Ana, Luis, Marta]` (ordenado)

Requisitos técnicos

Para usar `TreeSet`, los elementos deben ser:

- Comparables (`Comparable`)

  o
- Usar un `Comparator`

Necesita una regla de orden

Complejidad

Operación	Coste
`add`	O(log n)
`contains`	O(log n)

Más lento que `HashSet`, pero mantiene orden

3) Comparación global

Estructura	Orden	Duplicados	Complejidad
`HashSet`	No	No	O(1)
`LinkedHashSet`	Inserción	No	O(1)
`TreeSet`	Ordenado	No	O(log n)

4) Interpretación técnica

- `HashSet` → máximo rendimiento
- `LinkedHashSet` → equilibrio entre orden y rendimiento
- `TreeSet` → orden automático con mayor coste

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Orden Externo y Comportamiento Parametrizable.

1) Problema

No siempre queremos el orden natural (`Comparable`).

Ejemplo:

```
class Persona {
    String nombre;
    int edad;
}
```

¿Ordenar por nombre?

¿Ordenar por edad?

¿Orden inverso?

Necesitamos definir el criterio desde fuera.

2) Definición formal

`Comparator<T>` es una **interfaz funcional** que define:

```
int compare(T o1, T o2);
```

Contrato:

- < 0 → o1 va antes que o2
- = 0 → equivalentes en orden
- > 0 → o1 va después

3) Forma clásica (clase anónima)

```
Comparator<Persona> porEdad = new Comparator<>() {
    @Override
    public int compare(Persona p1, Persona p2) {
        return Integer.compare(p1.edad, p2.edad);
    }
};
```

Demasiado código.

4) Forma moderna (lambda)

```
Comparator<Persona> porEdad =
    (p1, p2) -> Integer.compare(p1.edad, p2.edad);
```

Más limpio: comportamiento como dato.

5) Uso práctico

En ordenación de lista:

```
lista.sort(porEdad);
```

En TreeSet:

```
Set<Persona> set =
    new TreeSet<>((p1, p2) ->
Integer.compare(p1.edad, p2.edad));
```

6) Versión aún más moderna

```
lista.sort(Comparator.comparing(p -> p.edad));
```

Con referencia a método:

```
lista.sort(Comparator.comparing(Persona::getEdad));
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Patrones de uso de Set: Pertenencia y Unicidad.

El uso de `Set` no se basa en "guardar elementos", sino en **garantizar unicidad** y realizar operaciones eficientes de **pertenencia**. Es especialmente útil cuando el problema se puede expresar en términos de conjuntos.

1) Eliminación de duplicados

Patrón típico: convertir una colección con repetidos en un conjunto único.

```
List<String> datos = Arrays.asList("Ana", "Luis", "Ana", "Marta");
Set<String> sinDuplicados = new HashSet<>(datos);
System.out.println(sinDuplicados);
```

Elimina automáticamente los duplicados
No requiere lógica adicional

2) Comprobación rápida de pertenencia

`Set` está optimizado para responder:

"¿Este elemento está en el conjunto?"

```
Set<String> usuarios = new HashSet<>();
usuarios.add("Ana");
usuarios.add("Luis");
if (usuarios.contains("Ana")) {
    System.out.println("Usuario válido");
}
```

Complejidad media → **O(1)**
Evita recorrer listas completas

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Patrones de uso de Set: Pertenencia y Unicidad.

3) Operaciones tipo conjunto

Aunque Java no tiene métodos directos como en matemáticas, se pueden implementar fácilmente:

Unión (A ∪ B)

```
Set<String> a = new HashSet<>(List.of("Ana", "Luis"));
Set<String> b = new HashSet<>(List.of("Luis", "Marta"));
a.addAll(b);
System.out.println(a);
```

Intersección (A ∩ B)

```
Set<String> a = new HashSet<>(List.of("Ana", "Luis"));
Set<String> b = new HashSet<>(List.of("Luis", "Marta"));
a.retainAll(b);
System.out.println(a);
```

Diferencia (A – B)

```
Set<String> a = new HashSet<>(List.of("Ana", "Luis"));
Set<String> b = new HashSet<>(List.of("Luis"));
a.removeAll(b);
System.out.println(a);
```

4) Interpretación técnica

- `HashSet` permite operaciones rápidas gracias al hashing
- La unicidad está garantizada por `hashCode()` y `equals()`
- Las operaciones de conjunto reutilizan métodos estándar

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Ejemplos con TreeSet.

```java
import java.util.Iterator;
import java.util.TreeSet;

public class TreeSet1 {
    public static void main(String[] args) {
        TreeSet<Integer> tree = new TreeSet<>();
        tree.add(12);
        tree.add(63);
        tree.add(34);
        tree.add(45);

        System.out.println("Los datos del árbol son los siguientes:");
        Iterator<Integer> iterator = tree.iterator();
        while (iterator.hasNext()) System.out.print(iterator.next() + " ");

        if (tree.isEmpty()) {
            System.out.print("El árbol está vacío.");
        } else {
            System.out.println("El tamaño del árbol es: " + tree.size());
        }

        System.out.println("Primer Dato del árbol: " + tree.first());
        System.out.println("Último dato del árbol: " + tree.last());

        if (tree.remove(45)) {
            System.out.println("El dato 45 es eliminado del árbol");
        } else {
            System.out.println("El dato 45 no existe!");
        }

        iterator = tree.iterator();
        while (iterator.hasNext()) System.out.print(iterator.next() + " ");

        System.out.println("\nEl tamaño del árbol es: " + tree.size());

        tree.clear();
        if (tree.isEmpty()) {
            System.out.print("El árbol está vacío.");
        } else {
            System.out.println("El tamaño del árbol es: " + tree.size());
        }
    }
}
```

```
run:
Los datos del árbol son los siguientes:
12 34 45 63 El tamaño del árbol es: 4
Primer Dato del árbol: 12
Último dato del árbol: 63
El dato 45 es eliminado del árbol
12 34 63
El tamaño del árbol es: 3
El árbol está vacío.BUILD SUCCESSFUL (total time: 0 seconds)
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Ejemplos con TreeSet.

```java
import java.util.Iterator;
import java.util.TreeSet;

public class PersonaTreeSet implements Comparable<PersonaTreeSet> {
  private String nombre;
  private int edad;

  public PersonaTreeSet(String nombre, int edad){
     this.nombre = nombre;
     this.edad = edad;
  }
    //damos el elemento a comparar con el orden natural.
    @Override
    public int compareTo(PersonaTreeSet otraPersona) {
        //si la variable es de String
       return (this.nombre.compareTo(otraPersona.nombre));
       //Si la variable es de enteros.
       // return (this.edad - otraPersona.edad));
    }

  @Override
    public String toString(){
       return "{"+nombre+","+edad+"}";
    }
```

```java
@Override
  public boolean equals(Object obj){
     if(this==obj) return true;
     if(obj==null) return false;
     if(getClass() != obj.getClass()) return false;
     PersonaTreeSet other = (PersonaTreeSet) obj;
     if(edad != other.edad)
         return false;
     if(nombre==null){
        if(other.nombre != null)
            return false;
     }else if(!nombre.equals(other.nombre))
         return false;
     return true;
     }

  public static void main(String [] args){
     TreeSet<PersonaTreeSet> set = new TreeSet<PersonaTreeSet>();
     set.add(new PersonaTreeSet("Jorge",30));
     set.add(new PersonaTreeSet("Carlos",48));
     set.add(new PersonaTreeSet("Miguel",29));
     set.add(new PersonaTreeSet("Carlos",46));
     set.add(new PersonaTreeSet("Jorge",25));

     Iterator<PersonaTreeSet> iterator = set.iterator();
     while(iterator.hasNext()){
       PersonaTreeSet persona = iterator.next();
       System.out.println(persona);
     }
  }
```

Orden natural: nombre.

```
run:
{Carlos,48}
{Jorge,30}
{Miguel,29}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Orden natural: edad:

```
run:
{Jorge,25}
{Miguel,29}
{Jorge,30}
{Carlos,46}
{Carlos,48}
BUILD SUCCESSFUL (total tir}
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Set: HashSet, LinkedHashSet y TreeSet.

Ejemplos con LinkedHashSet.

```java
import java.util.LinkedHashSet;
public class LinkedHashSet1{
    public static void main(String[] args){
        LinkedHashSet<String> linkedset;
        linkedset = new LinkedHashSet<String>();
        linkedset.add("A");
        linkedset.add("B");
        linkedset.add("C");
        linkedset.add("D");
        linkedset.add("A");
        linkedset.add("E");
        System.out.println("Tamaño = " + linkedset.size());
        System.out.println("Valores:" + linkedset);
        System.out.println("Borrar D del LinkedHashSet: " + linkedset.remove("D"));
        System.out.println("Remover Z que no existe: " + linkedset.remove("Z"));
        System.out.println("Comprobamos si A está presente: "+linkedset.contains("A"));
        System.out.println("Mostramos el LinkedHashSet: " + linkedset);
        }
}
```

- ProyectoPrimero (run)

```
run:
Tamaño = 5
Valores:[A, B, C, D, E]
Borrar D del LinkedHashSet: true
Remover Z que no existe: false
Comprobamos si A está presente: true
Mostramos el LinkedHashSet: [A, B, C, E]
BUILD SUCCESSFUL (total time: 0 seconds)
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

Map: Modelo de Datos y Operaciones Fundamentales.

`Map` no es una colección de elementos, sino una estructura de **asociación clave → valor**. Cada dato se almacena junto a una clave única que permite acceder a él de forma directa.

1) Modelo conceptual de Map

Un `Map<K,V>` representa:

- Un conjunto de pares **clave → valor**
- Claves **únicas**
- Valores que pueden repetirse

Ejemplo:

```
Map<String, Integer> edades = new HashMap<>();
edades.put("Ana", 25);
edades.put("Luis", 30);
edades.put("Ana", 40);
```

Resultado: `{"Ana"=40, "Luis"=30}`

El valor asociado a la clave se sobrescribe. La clave es única.

2) Operaciones fundamentales

- **Inserción / actualización:** `map.put(clave, valor);`
- **Acceso:** `map.get(clave);`
- **Comprobación:** `map.containsKey(clave);`
- **Eliminación:** `map.remove(clave);`
- **Tamaño:** `map.size();`

3) Interpretación técnica

`Map` está diseñado para:

- Acceso directo mediante clave
- Evitar búsquedas lineales
- Asociar información de forma estructurada

Es equivalente a un **diccionario o índice**

4) Comportamiento clave

- Las claves son únicas
- `put()` reemplaza el valor si la clave existe
- `get()` devuelve `null` si la clave no existe

Esto obliga a validar en muchos casos

5) Ejemplo típico de uso

Contar apariciones:

```
Map<String, Integer> contador = new HashMap<>();
for (String palabra : texto) {
    contador.put(palabra,contador.getOrDefault(palabra, 0) + 1);
}
```

Patrón muy importante en programación real

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

Patrones de uso de Map: Acceso y Acumulación.

El valor de `Map` no está en almacenar datos, sino en **acceder a ellos por clave** y en **acumular información de forma eficiente**. Estos patrones aparecen constantemente en programación real.

1) Contador de elementos (frecuencias)

Patrón clásico: contar apariciones usando la clave como identificador.

```
String[] palabras =
{"java", "map", "java", "set", "map", "java"};
Map<String, Integer> contador = new HashMap<>();
for (String p : palabras) {
    contador.put(p, contador.getOrDefault(p, 0) + 1);
}
System.out.println(contador);
```

`getOrDefault` evita comprobar si la clave existe

Patrón muy frecuente en procesamiento de datos

2) Índice (clave → objeto)

Uso de `Map` como estructura de acceso directo:

```
Map<String, String> usuarios = new HashMap<>();
usuarios.put("u1", "Ana");
usuarios.put("u2", "Luis");
// acceso directo
String usuario = usuarios.get("u1");
```

Sustituye búsquedas lineales

Acceso eficiente → **O(1)**

3) Iteración correcta: entrySet()

Forma recomendada de recorrer un `Map`:

```
for (Map.Entry<String, Integer> e :
contador.entrySet()) {
    System.out.println(e.getKey() + " -> " +
    e.getValue());
}
```

Acceso directo a clave y valor

Evita búsquedas adicionales

4) Error típico (menos eficiente)

```
for (String k : contador.keySet()) {
    System.out.println(k + " -> " +
    contador.get(k));
}
```

Hace una búsqueda por cada clave

Menos eficiente

5) Interpretación técnica

- `Map` optimiza acceso por clave mediante hashing
- Permite construir estructuras derivadas (contadores, índices)
- Es base de muchas soluciones reales

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

Ejemplo Completo: Acceso por Clave (índice).

Este ejemplo muestra cómo usar un **`Map`** como estructura de acceso directo, evitando recorridos innecesarios.

Código

```
Map<String, String> usuarios = new HashMap<>();

usuarios.put("u1", "Ana");
usuarios.put("u2", "Luis");
usuarios.put("u3", "Marta");

// acceso directo
String usuario = usuarios.get("u2");

System.out.println(usuario);
```

Resultado esperado

```
Luis
```

Qué está ocurriendo

- Cada usuario tiene un **identificador único (clave)**
- El **`Map`** permite acceder directamente al valor
- No es necesario recorrer ninguna estructura

Acceso inmediato mediante clave

Qué se está trabajando

- **Modelo clave → valor**
- **Acceso directo (lookup)**
- Sustitución de búsquedas lineales

Interpretación técnica

Sin **`Map`**, habría que hacer:

```
for (Usuario u : lista) {
    if (u.getId().equals("u2")) {
        return u;
    }
}
```

Complejidad: **O(n)**

Con **`Map`**: `usuarios.get("u2");`

Complejidad: **O(1)**

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

HashMap: Modelo Interno, Hashing y Complejidad

`HashMap` es la implementación más utilizada de **`Map`**. Su objetivo es permitir **acceso rápido por clave**, evitando recorridos completos. Internamente utiliza una **tabla hash**, el mismo principio que **`HashSet`**.

1) Modelo interno: tabla hash de pares clave → valor

Un **`HashMap`** se basa en:

- Un array de buckets
- Cada bucket contiene entradas (**`Entry<K,V>`**)

Cada entrada almacena:

- clave
- valor
- hash
- referencia al siguiente (en caso de colisión)

Representación conceptual:

```
índice:   0     1     2     3     4
        [ ]   [K1]  [ ]   [K2]  [K3 -> K4]
```

Varias claves pueden caer en el mismo bucket → colisiones

2) Proceso de inserción (`put`)

```
map.put(clave, valor);
```

Pasos:

1.Se calcula **`hashCode()`** de la clave

2.Se obtiene el índice del bucket

3.Se comprueba si la clave ya existe (**`equals`**)

1. Si existe → se **reemplaza el valor**
2. Si no → se añade nueva entrada

3) Búsqueda (`get`)

```
map.get(clave);
```

- Calcula el hash
- Accede directamente al bucket
- Busca dentro del bucket (si hay colisión)

No recorre toda la estructura

4) Complejidad de operaciones

Operación	Coste medio	Justificación
`put`	O(1)	acceso directo por hash
`get`	O(1)	búsqueda directa
`remove`	O(1)	acceso directo

En caso de muchas colisiones:

- Puede degradar a **O(n)**

5) Factores críticos de rendimiento

- Implementación de `hashCode()`
- Uso correcto de `equals()`
- Distribución uniforme de hashes
- Factor de carga (load factor)

Mal diseño → pérdida de eficiencia

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

LinkedHashMap y TreeMap: orden, estructura y comparación.

En `Map`, el **orden de los elementos no es intrínseco** salvo que la implementación lo defina.
`LinkedHashMap` y `TreeMap` introducen orden, pero con **criterios y costes distintos**.

1) LinkedHashMap — orden de inserción

```
Map<String, Integer> m = new LinkedHashMap<>();
m.put("Ana", 1);
m.put("Luis", 2);
m.put("Marta", 3);
System.out.println(m);
```

Resultado: `{Ana=1, Luis=2, Marta=3}`

Qué ocurre

- Mantiene el orden en que se insertan las entradas
- Internamente: **tabla hash + lista doblemente enlazada**
- Permite acceso rápido por clave (como `HashMap`)

2) TreeMap — orden natural / comparador

```
Map<String, Integer> m = new TreeMap<>();
m.put("Luis", 2);
m.put("Ana", 1);
m.put("Marta", 3);
System.out.println(m);
```

Resultado: `{Ana=1, Luis=2, Marta=3}`

Qué ocurre

- Ordena las claves automáticamente
- Internamente: **árbol balanceado (Red-Black Tree)**

3) Comparación estructural y de comportamiento

Característica	LinkedHashMap	TreeMap
Orden	Inserción	Ordenado
Estructura	Hash + lista	Árbol balanceado
Acceso (get/put)	O(1)	O(log n)
Iteración	Orden de inserción	Orden ordenado
Requiere Comparable	❌ No	✓ Sí

4) Interpretación técnica

- `LinkedHashMap` añade **orden sin penalizar apenas el acceso**
- `TreeMap` introduce **orden lógico (clasificación)** a costa de rendimiento

El orden en `TreeMap` tiene coste estructural
El orden en `LinkedHashMap` es “memoria + enlaces”

- El orden depende de:
  - orden natural (`Comparable`)
  - o un `Comparator`

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

Ejemplo de HashMap

```java
import java.util.Scanner;
import java.util.HashMap;
import java.util.Map;
public class HashMap1 {
    public static void main(String[] args) {
        HashMap<Integer, String> m = new HashMap<Integer, String>();
        m.put(924, "Amalia Núñez");
        m.put(921, "Cindy Nero");
        m.put(700, "César Vázquez");
        m.put(219, "Víctor Tilla");
        m.put(537, "Alan Brito");
        m.put(605, "Esteban Quito ");
        System.out.println("Los elementos de m son: \n" + m);
        System.out.println("Todas las entradas del diccionario extraídas con entrySet:");
        System.out.println(m.entrySet());
        System.out.println("\nEntradas del diccionario extraídas una a una:");
        for (Map.Entry pareja: m.entrySet()) {
            System.out.println(pareja);
            System.out.print(pareja.getKey() + "\t");
            System.out.println(pareja.getValue());
        }
        System.out.print("Por favor, introduzca un código: ");
        Scanner sc = new Scanner(System.in);
        int codigoIntroducido = sc.nextInt();
        if (m.containsKey(codigoIntroducido)) {
            System.out.print("El código " + codigoIntroducido + " corresponde a ");
            System.out.println(m.get(codigoIntroducido));
        } else {
            System.out.print("El código introducido no existe.");
        }
    }
}
```

```
run:
Los elementos de m son:
{921=Cindy Nero, 537=Alan Brito, 219=Víctor Tilla, 924=Amalia Núñez, 700=César Vázquez, 605=Esteban Quito }
Todas las entradas del diccionario extraídas con entrySet:
[921=Cindy Nero, 537=Alan Brito, 219=Víctor Tilla, 924=Amalia Núñez, 700=César Vázquez, 605=Esteban Quito ]

Entradas del diccionario extraídas una a una:
921=Cindy Nero
921     Cindy Nero
537=Alan Brito
537     Alan Brito
219=Víctor Tilla
219     Víctor Tilla
924=Amalia Núñez
924     Amalia Núñez
700=César Vázquez
700     César Vázquez
605=Esteban Quito
605     Esteban Quito
Por favor, introduzca un código: 921
El código 921 corresponde a Cindy Nero
BUILD SUCCESSFUL (total time: 7 seconds)
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

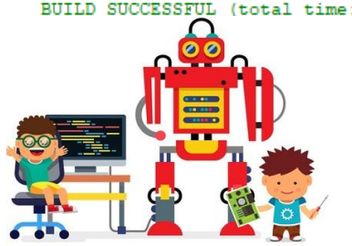
Map: HashMap, LinkedHashMap y TreeMap.

Ejemplos de TreeMap.

```java
import java.util.*;
import java.util.concurrent.*;

public class TreeMap1 {
    static void Example1stConstructor() {
        TreeMap<Integer, String> tree_map = new TreeMap<Integer, String>();
        tree_map.put(10, "Geeks");
        tree_map.put(15, "4");
        tree_map.put(20, "Geeks");
        tree_map.put(25, "Welcomes");
        tree_map.put(30, "You");
        System.out.println("TreeMap: " + tree_map);
        }

        public static void main(String[] args){
            System.out.println("TreeMap using "
                                + "TreeMap() constructor:\n");
            Example1stConstructor();
        }
}
```

- ProyectoPrimero (run)

```
run:
TreeMap using TreeMap() constructor:

TreeMap: {10=Geeks, 15=4, 20=Geeks, 25=Welcomes, 30=You}
BUILD SUCCESSFUL (total time: 0 seconds)
```



```java
import java.util.*;
import java.util.concurrent.*;
class Student {
    int rollno;
    String name, address;
    public Student(int rollno, String name,
                   String address){
        this.rollno = rollno;
        this.name = name;
        this.address = address;
    }
    @Override
    public String toString(){
        return this.rollno + " "
            + this.name + " "
            + this.address;
    }
}
class Sortbyroll implements Comparator<Student> {
    @Override
    public int compare(Student a, Student b) {
        return a.rollno - b.rollno;
    }
}
```

```java
public class TreeMap2 {
    static void Example2ndConstructor(){
        TreeMap<Student, Integer> tree_map;
        tree_map = new TreeMap<Student, Integer>(new Sortbyroll());
        tree_map.put(new Student(111, "bbbb", "london"), 2);
        tree_map.put(new Student(131, "aaaa", "nyc"), 3);
        tree_map.put(new Student(121, "cccc", "jaipur"), 1);
        System.out.println("TreeMap: " + tree_map);
    }

    public static void main(String[] args) {
        System.out.println("TreeMap using "
                           + "TreeMap(Comparator)"
                           + " constructor:\n");
        Example2ndConstructor();
    }
}
```

```
run:
TreeMap using TreeMap(Comparator) constructor:

TreeMap: {111 bbbb london=2, 121 cccc jaipur=1, 131 aaaa nyc=3}
BUILD SUCCESSFUL (total time: 0 seconds)
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Map: HashMap, LinkedHashMap y TreeMap.

Ejemplo de LinkedhashMap.

```java
import java.util.*;
class LinkedHashMap1{
    public static void main(String args[]){
        LinkedHashMap<Integer, String> map = new LinkedHashMap<Integer, String>();
        map.put(100,"Amit");
        map.put(101,"Vijay");
        map.put(102,"Rahul");
        System.out.println("Keys: "+map.keySet());
        System.out.println("Values: "+map.values());
        System.out.println("Key-Value pairs: "+map.entrySet());
        for(Map.Entry m:map.entrySet()){
            System.out.println(m.getKey()+" "+m.getValue());
        }
        map.remove(102);
        System.out.println("After invoking remove() method: "+map);
    }
}
```

```
- ProyectoPrimero (run)
run:
Keys: [100, 101, 102]
Values: [Amit, Vijay, Rahul]
Key-Value pairs: [100=Amit, 101=Vijay, 102=Rahul]
100 Amit
101 Vijay
102 Rahul
After invoking remove() method: {100=Amit, 101=Vijay}
BUILD SUCCESSFUL (total time: 8 seconds)
```

Gestión de Datos en Memoria

Colecciones Principales: List, Set y Map.

Estructura / Implementación	Duplicados	Orden en iteración	Acceso típico	Coste típico búsqueda	Coste típico inserción	Notas de uso
Array (T[])	Sí	Sí (índice)	Índice	O(n) (si buscas)	N/A (tamaño fijo)	Tamaño fijo; útil para datos densos/buffers
ArrayList	Sí	Sí (inserción)	Índice (get(i))	O(n) (contains)	add final O(1) amort.; add/remove en medio O(n)	Lista "por defecto" si necesitas índice y buen rendimiento general
LinkedList	Sí	Sí (inserción)	Extremos / secuencial	O(n)	add/remove extremos O(1); acceso por índice O(n)	Útil como cola/deque; mala para get(i) frecuente
HashSet	No	No garantizado	Pertenencia (contains)	O(1) medio (peor O(n))	O(1) medio	Requiere contrato equals/hashCode correcto; máximo rendimiento sin orden
LinkedHashSet	No	Sí (inserción)	Pertenencia (contains)	O(1) medio	O(1) medio	Mantiene orden de inserción con pequeño overhead
TreeSet	No	Sí (orden natural/comparator)	Pertenencia / rango	O(log n)	O(log n)	Ordenado siempre; requiere Comparable o Comparator
HashMap	Claves no / valores sí	No garantizado	Por clave (get/put)	O(1) medio (peor O(n))	O(1) medio	"Diccionario" rápido; claves con equals/hashCode correctos
LinkedHashMap	Claves no / valores sí	Sí (inserción)	Por clave (get/put)	O(1) medio	O(1) medio	Igual que HashMap pero iteración en orden de inserción
TreeMap	Claves no / valores sí	Sí (claves ordenadas)	Por clave (get/put) + orden	O(log n)	O(log n)	Mapa ordenado; requiere Comparable/Comparator; útil para listados ordenados y rangos

- Necesito índice/posición → List (normalmente ArrayList).
- Necesito unicidad → Set (HashSet si no importa el orden, LinkedHashSet si importa, TreeSet si debe ir ordenado).
- Necesito buscar por id/clave → Map (HashMap / LinkedHashMap / TreeMap según orden).
- Necesito orden "automático" por criterio → TreeSet / TreeMap (+Comparable/Comparator).

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Interfaces Funcionales (base conceptual).

¿Qué es una Interfaz Funcional?

Una **interfaz funcional** es una interfaz que define **exactamente un único método abstracto**.
Este tipo de interfaz permite representar **comportamientos** que pueden pasarse como parámetro.

1) Definición formal

Una interfaz funcional es aquella que contiene un solo método abstracto y puede ser implementada mediante expresiones lambda.

2) Ejemplo básico

```
interface Operacion {
    int aplicar(int a, int b);
}
```

Define una operación genérica sobre dos valores

3) Uso clásico (sin lambda)

```
Operacion suma = new Operacion() {
    public int aplicar(int a, int b) {
        return a + b;
    }
};
System.out.println(suma.aplicar(3, 4));
```

Qué está ocurriendo

- Se define un comportamiento (sumar)
- Se implementa mediante una clase anónima
- Se ejecuta a través del método de la interfaz

Estamos tratando **funciones como datos**

4) Características clave

- Un único método abstracto
- Puede tener métodos `default` o `static`
- Base de la programación funcional en Java

5) Relación con el siguiente paso

Las interfaces funcionales permiten simplificar su implementación mediante expresiones lambda.

Idea clave

Una interfaz funcional representa una acción que puede ser pasada, almacenada y ejecutada.

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Clases Anónimas

Clases Anónimas: Implementación Inline de Comportamientos.

Las **clases anónimas** permiten implementar una interfaz (o clase abstracta) **en el mismo lugar donde se usa**, sin definir una clase con nombre.

1) Motivación

Partimos de una interfaz funcional:

```
interface Operacion {
    int aplicar(int a, int b);
}
```

Sin clases anónimas, necesitaríamos:

```
class Suma implements Operacion {
    public int aplicar(int a, int b) {
        return a + b;
    }
}
```

Demasiado código para algo simple

2) Solución: clase anónima

```
Operacion suma = new Operacion() {
    public int aplicar(int a, int b) {
        return a + b;
    }
};

System.out.println(suma.aplicar(3, 4));
```

Qué está ocurriendo

- Se crea una implementación **sin nombre**
- Se define directamente donde se necesita
- Se evita crear una clase adicional

Se reduce la sobrecarga estructural

3) Ventajas

- Código más localizado
- Evita clases innecesarias
- Útil para implementaciones puntuales

4) Limitaciones

- Sintaxis **verbosa**
- Difícil de leer si crece
- No escala bien en lógica compleja

5) Interpretación técnica

- El compilador genera una clase interna
- Se instancia en tiempo de ejecución
- Se usa como cualquier objeto

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Clases Anónimas

Clases Anónimas: Implementación Inline de Comportamientos.

6) Transición conceptual

Las clases anónimas resuelven el problema de la verbosidad,
pero aún son demasiado pesadas para expresar comportamientos simples.
De aquí surgen las **expresiones lambda**

Idea clave

Una clase anónima permite definir comportamiento en el punto de uso,
pero con una sintaxis todavía demasiado extensa.

Runnable (tareas concurrentes)

```
Thread t = new Thread(new Runnable() {
    public void run() {
        System.out.println("Ejecutando tarea
en hilo");
    }
});
t.start();
```

Qué se está trabajando

- Implementación de `Runnable`
- Definición de comportamiento ejecutable
- Uso en concurrencia

Comparator (ordenación personalizada)

```
List<String> nombres = List.of("Ana", "Luis",
"Marta", "Alberto");

List<String> copia = new ArrayList<>(nombres);

Collections.sort(copia, new Comparator<String>() {
    public int compare(String a, String b) {
        return a.length() - b.length();
    }
});

System.out.println(copia);
```

Qué se está trabajando

- Implementación inline de una interfaz (`Comparator`)
- Paso de comportamiento como parámetro
- Personalización del criterio de ordenación

Caso real muy frecuente

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Clases Anónimas

Clases Anónimas: Implementación Inline de Comportamientos.

ActionListener (GUI Swing)

```
JButton boton = new JButton("Pulsar");

boton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e)
{
        System.out.println("Botón pulsado");
    }
});
```

Qué se está trabajando

- Manejo de eventos
- Código asociado a interacción del usuario
- Implementación inline de listeners

Caso clásico en interfaces gráficas

Filtro personalizado

```
interface Filtro {
    boolean aceptar(int x);
}
Filtro positivos = new Filtro() {
    public boolean aceptar(int x) {
        return x > 0;
    }
};
System.out.println(positivos.aceptar(5));
```

Qué se está trabajando

- Definición de criterio dinámico
- Separación de lógica
- Base conceptual para `filter` en streams

En todos los ejemplos ocurre lo mismo:
Se define un comportamiento **sin crear una clase explícita**

Mensaje clave para el alumno

Todos estos ejemplos tienen un problema común:

- Mucho código
- Mucha sintaxis repetitiva
- Poco expresivo

Esto justifica la aparición de **lambda**

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Expresiones Lambda

Expresiones Lambda: Simplificación de Clases Anónimas.

Las **expresiones lambda** permiten implementar interfaces funcionales de forma **concisa**, eliminando la verbosidad de las clases anónimas.

1) Forma general

```
(parámetros) -> expresión
o
(parámetros) -> {
    instrucciones;
}
```

Representan directamente el cuerpo del método abstracto

2) Equivalencia con clase anónima

Clase anónima:

```
Operacion suma = new Operacion() {
    public int aplicar(int a, int b) {
        return a + b;
    }
};
```

Lambda equivalente:

```
Operacion suma = (a, b) -> a + b;
```

Qué cambia

- Se elimina:
  - `new`
  - nombre de la interfaz
  - cabecera del método

3) Ejemplo real: Comparator

Clase anónima:

```
Collections.sort(lista, new
Comparator<String>() {
    public int compare(String a, String b) {
        return a.length() - b.length();
    }
});
```

Lambda:

```
Collections.sort(lista, (a, b) -> a.length() -
b.length());
```

4) Reglas importantes

- Solo funcionan con **interfaces funcionales**
- El tipo se **infiere automáticamente**
- Si hay una sola expresión → no necesita `return`
- Si hay varias instrucciones → usar `{}`

5) Interpretación técnica

- La lambda no crea una clase como tal
- Se traduce internamente a una forma optimizada
- Representa comportamiento como valor

6) Ventajas

- Código más corto
- Mayor legibilidad
- Facilita composición de operaciones

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Expresiones Lambda

Expresiones Lambda: Con Interfaces Propias.

1) Interfaz funcional base

```
interface Operacion {
    int aplicar(int a, int b);
}
```

Método abstracto:

```
int aplicar(int a, int b);
```

2) Clase anónima (punto de partida)

```
Operacion suma = new Operacion() {
    public int aplicar(int a, int b) {
        return a + b;
    }
};
```

3) Lambda equivalente

```
Operacion suma = (a, b) -> a + b;
```

Interpretación exacta

La lambda:

```
(a, b) -> a + b
```

equivale a:

```
int aplicar(int a, int b) {
    return a + b;
}
```

Está implementando el método `aplicar`

4) Otro ejemplo (condición)

```
interface Condicion {
    boolean evaluar(int x);
}
```

Clase anónima:

```
Condicion positivo = new Condicion() {
    public boolean evaluar(int x) {
        return x > 0;
    }
};
```

Lambda:

```
Condicion positivo = x -> x > 0;
```

Equivalencia con

```
boolean evaluar(int x) {
    return x > 0;
}
```

5) Lambda con bloque (varias instrucciones)

```
Operacion op = (a, b) -> {
    int resultado = a + b;
    return resultado * 2;
};
```

Aquí sí es obligatorio `return`

6) Reglas clave (rigurosas)

- Solo funcionan con **interfaces funcionales (1 método abstracto)**
- Los tipos se **infieren** (no se escriben)
- Si hay una sola expresión → retorno implícito
- Si hay bloque `{}` → requiere `return`

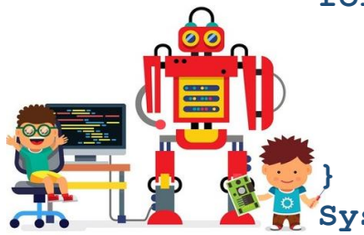
Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Expresiones Lambda

Expresiones Lambda: Con Interfaces Propias.

```
import java.util.ArrayList;
import java.util.List;

public class LambdasBasico {
    // Interfaz funcional propia 1:
    interface Condicion {
        boolean evaluar(int x);
    }
    // Interfaz funcional propia 2:
    interface Operacion {
        int aplicar(int a, int b);
    }
    public static void main(String[] args) {
        // Datos de ejemplo
        List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);
        // 1) Lambda sobre interfaz propia (Condicion)
        Condicion esPar = x -> x % 2 == 0;
        // Uso: filtrar manualmente con for-each
        List<Integer> pares = new ArrayList<>();
        for (int n : numeros) {
            if (esPar.evaluar(n)) {
                pares.add(n);
            }
        }
        System.out.println("Pares: " + pares);
```



```
        // 2) Lambda sobre interfaz propia (Operacion)
        Operacion suma = (a, b) -> a + b;
        Operacion producto = (a, b) -> a * b;
        // Uso: reducción manual con bucle clásico
        int totalSuma = 0;
        for (int n : pares) {
            totalSuma = suma.aplicar(totalSuma, n);
        }
        System.out.println("Suma de pares: " + totalSuma);
        int totalProducto = 1;
        for (int n : pares) {
            totalProducto = producto.aplicar(totalProducto, n);
        }
        System.out.println("Producto de pares: " +
        totalProducto);
    }
}
```

Qué conceptos usa (y por qué encaja con lo visto)

1. Interfaz funcional propia
   - `Condicion` con `evaluar(int)`
   - `Operacion` con `aplicar(int,int)`
2. Lambda como implementación directa del método abstracto
   - `x -> x % 2 == 0` implementa `evaluar(x)`
   - `(a,b) -> a + b` implementa `aplicar(a,b)`
3. Colecciones y bucles normales
   - `ArrayList` para construir resultados
   - `for-each` / `for` para recorrer
   - Sin utilidades “nuevas” ni APIs adicionales

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Expresiones Lambda

Expresiones Lambda: Con Interfaces de API Java.

Ejemplo 1 — Predicate (java.util.function)

Interfaz: `boolean test(T t);`

Uso:

```
import java.util.function.Predicate;
Predicate<Integer> esPar = x -> x % 2 == 0;
System.out.println(esPar.test(4)); // true
System.out.println(esPar.test(5)); // false
```

La lambda implementa `test(x)`

Ejemplo 2 — Function (java.util.function)

Interfaz: `R apply(T t);`

Uso:

```
import java.util.function.Function;
Function<Integer, Integer> cuadrado = x -> x * x;
System.out.println(cuadrado.apply(5)); // 25
```

Entrada → salida

Ejemplo 3 — Consumer (java.util.function)

Interfaz: `void accept(T t);`

Uso:

```
import java.util.function.Consumer;
Consumer<String> imprimir = s ->
System.out.println(s);
imprimir.accept("Hola mundo");
```

Acción sin retorno

Ejemplo 4 — Supplier (java.util.function)

Interfaz: `T get();`

Uso:

```
import java.util.function.Supplier;
Supplier<Double> aleatorio = () -> Math.random();
System.out.println(aleatorio.get());
```

No recibe nada, genera un valor

Ejemplo 5 — Comparator (java.util)

Interfaz: `int compare(T o1, T o2);`

Uso:

```
import java.util.*;
List<String> nombres;
nombres=new ArrayList<>(List.of("Ana","Luis","Marta"));
Collections.sort(nombres,(a,b)->a.length()-b.length());
System.out.println(nombres);
```

Ordenación personalizada

Ejemplo 6 — Runnable (java.lang)

Interfaz: `void run();`

Uso:

```
Runnable tarea = ()-> System.out.println("Ejecutando");
new Thread(tarea).start();
```

Código ejecutable en hilo

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Expresiones Lambda

Patrón de Uso: Comportamiento Aplicado a Colecciones.

Esta diapositiva no introduce nuevas APIs, sino que **formaliza el modelo de uso** que ya hemos visto: aplicar comportamiento sobre colecciones sin escribir la lógica dentro del bucle.

1) Modelo tradicional (imperativo)

```
List<Integer> datos = List.of(1, 2, 3, 4, 5, 6);

for (int x : datos) {
    if (x % 2 == 0) {
        System.out.println(x);
    }
}
```

Características

- El bucle controla todo
- La lógica está incrustada
- Baja reutilización

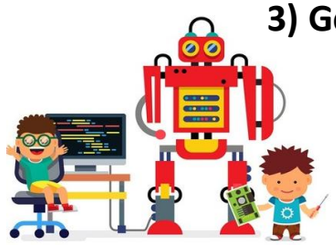
2) Modelo con lambdas (delegación de comportamiento)

```
datos.forEach(x -> System.out.println(x));
```

La colección gestiona el recorrido

La lambda define la acción

3) Generalización del modelo



```
Colección + interfaz funcional + lambda → operación
```

Más formalmente:

```
Estructura de datos + comportamiento parametrizado → ejecución
```

4) Tipos de comportamiento implicados (API Java)

Tipo	Interfaz	Método	Semántica
Acción	Consumer	`accept(T)`	hacer algo
Condición	Predicate	`test(T)`	decidir
Transformación	Function	`apply(T)`	transformar
Comparación	Comparator	`compare(T,T)`	ordenar

Las lambdas implementan estos métodos

5) Descomposición técnica

Ejemplo: `datos.removeIf(x -> x % 2 == 0);`

Equivale a:

```
boolean test(Integer x) {
    return x % 2 == 0;
}
```

La colección invoca ese comportamiento internamente

6) Qué cambia realmente

Antes	Ahora
Control manual del flujo	Control delegado
Código repetitivo	Código parametrizado
Baja abstracción	Alta expresividad

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: concepto y primer pipelin

Streams: qué son y cómo se Leen (pipeline).

Un **Stream** es una abstracción para procesar datos de forma **declarativa**. No es una estructura de almacenamiento: es un **flujo de elementos** sobre el que se encadenan operaciones.

1) Qué es un Stream (definición operativa)

- Se obtiene de una fuente (colección, array, etc.)
- Se aplican operaciones intermedias (transforman el flujo)
- Se finaliza con una operación terminal (produce resultado)

2) Estructura de un pipeline

```
fuente → operaciones intermedias → operación terminal
```

Ejemplo conceptual:

```
List<Integer> → filter → map → collect
```

3) Ejemplo completo (primer pipeline)

```
import java.util.List;
import java.util.stream.Collectors;
List<Integer> datos = List.of(1, 2, 3, 4, 5, 6);
List<Integer> resultado = datos.stream()
    .filter(x -> x % 2 == 0)   //quedarse con pares
    .map(x -> x * x)   //transformar: cuadrado
    .collect(Collectors.toList());
                        // materializar en lista
System.out.println(resultado);
```

Salida esperada: `[4, 16, 36]`

4) Cómo se lee correctamente (muy importante)

No se lee como pasos “de programación”, sino como consulta: “De estos datos, quédate con los pares, conviértelos en su cuadrado y devuélveme la lista.”

El foco está en el **qué**, no en el **cómo**.

5) Propiedades técnicas esenciales

- Un Stream es **perezoso (lazy)**:
  las operaciones intermedias no ejecutan nada por sí solas
- La ejecución ocurre cuando aparece una operación terminal (`collect`, `sum`, `count`, etc.)
- No modifica la colección original (salvo efectos laterales explícitos)

6) Tipos de operaciones

Intermedias (devuelven Stream):

- `filter, map, sorted, distinct, limit`

Terminales (cierran el flujo):

- `collect, forEach, count, reduce, sum`

Idea clave

Streams permiten describir un procesamiento encadenado sobre datos, con evaluación perezosa y operaciones declarativas.

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Modelo de Ejecución.

Cómo se ejecuta un Stream: Evaluación perezosa y pipeline.

Los Streams **no ejecutan cada operación por separado**, sino que construyen un **pipeline** que se evalúa **de forma perezosa (lazy)** cuando aparece una operación terminal.

1) Ejemplo base

```
List<Integer> datos = List.of(1, 2, 3, 4, 5, 6);
datos.stream()
      .filter(x -> x % 2 == 0)
      .map(x -> x * x)
      .forEach(x -> System.out.println(x));
```

2) Cómo se podría pensar (incorrecto)

❌ Modelo mental erróneo:

```
1) filtrar todos los elementos
2) luego mapear todos
3) luego imprimir todos
```

Esto implicaría estructuras intermedias (listas temporales)

3) Cómo se ejecuta realmente (correcto)

✓ Evaluación por elemento:

```
1 → filter ❌ (se descarta)
2 → filter ✓ → map → forEach → imprime 4
3 → filter ❌
4 → filter ✓ → map → forEach → imprime 16
...
```

Cada elemento recorre todo el pipeline antes de pasar al siguiente

4) Consecuencia clave: evaluación perezosa

- Las operaciones intermedias **no se ejecutan solas**
- Se activan solo cuando hay una operación terminal
- No se crean estructuras intermedias

5) Ventajas del modelo

- Menor consumo de memoria
- Mayor eficiencia
- Posibilidad de optimización interna
- Facilita paralelización

6) Conceptos técnicos importantes

- **Pipeline**: cadena de operaciones
- **Lazy evaluation**: ejecución diferida
- **Internal iteration**: el Stream controla el recorrido

7) Comparación con enfoque tradicional

Imperativo	Streams
Control manual	Control interno
Iteración explícita	Iteración implícita
Paso a paso	Flujo continuo

Un Stream procesa los datos elemento a elemento a través de todo el pipeline, no operación a operación.

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Intermedias.

Tranformación con map().

La operación `map()` permite transformar cada elemento del Stream en otro valor.

1) Definición

`map: T → R`

Recibe un elemento

Devuelve otro transformado

2) Ejemplo básico

```
List<Integer> datos = List.of(1, 2, 3, 4);

List<Integer> cuadrados = datos.stream()
        .map(x -> x * x)
        .toList();
```

Resultado `[1, 4, 9, 16]`

3) Interpretación

```
for (int x : datos) {
    resultado.add(x * x);
}
```

`map` sustituye este patrón

4) Cambio de tipo (muy importante)

```
List<String> textos = List.of("Ana", "Luis");
List<Integer> longitudes = textos.stream()
        .map(s -> s.length())
        .toList();
```

Qué ocurre

`String → Integer`

`map` puede cambiar el tipo del Stream

5) Relación con interfaces funcionales

`map()` usa:

```
R apply(T t)    // Function<T,R>
```

6) Ejemplo con referencia a método

```
textos.stream()
        .map(String::length)
        .forEach(System.out::println);
```

7) Idea clave

`map()` transforma los datos sin modificar la colección original.

- Es la operación más importante junto con `filter`
- Insistir en:
  - transformación
  - cambio de tipo
- Preparar para combinaciones (`filter + map`)

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Intermedias.

Sorted(): ordenación declarativa en un Stream.

1) Qué hace

`sorted()` es una operación intermedia.
Devuelve un nuevo `Stream` con los elementos ordenados.
Puede usarse:

- Sin argumentos → orden natural (`Comparable`)
- Con `Comparator` → orden personalizado

No modifica la colección original.

2) Ejemplo con orden natural

```
import java.util.List;
List<Integer> datos = List.of(5, 2, 9, 1, 4);
List<Integer> ordenados = datos.stream()
    .sorted()
    .toList();
System.out.println(ordenados); //[1, 2, 4, 5, 9]
```

Interpretación:

- Se crea un Stream
- Se ordena según orden natural
- Se materializa con `toList()`

Equivalente imperativo:

```
List<Integer> copia = new ArrayList<>(datos);
Collections.sort(copia);
```

3) Ejemplo con Comparator (orden personalizado)

```
List<String> nombres = List.of("Ana","Luis","Marta");
List<String> porLongitud;
porLongitud = nombres.stream().sorted((a, b) ->
    Integer.compare(a.length(),b.length())).toList();
System.out.println(porLongitud); //[Ana,Luis,Marta]
```

Versión más expresiva:

```
List<String> porLongitud = nombres.stream()
    .sorted(Comparator.comparing(String::length))
    .toList();
```

4) Idea clave

- `sorted()` no cambia la colección original.
- Es declarativo: describe el criterio, no el proceso.
- Se integra en el pipeline.
- Requiere `Comparable` o `Comparator`.

5) Observación técnica importante

`sorted()` es intermedia pero **necesita recorrer todos los elementos** antes de producir resultado.
No es “short-circuiting” como `limit()`.
Complejidad aproximada: O(n log n).

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Intermedias.

Distinct(): eliminar duplicados lógicos en un Stream.

1.Qué hace

`distinct()` elimina elementos repetidos en el flujo.
Importante: determina "duplicado" usando `equals()` y `hashCode()`.

2.Ejemplo (String)

```
import java.util.List;
List<String> nombres = List.of("Ana", "Luis",
"Ana", "Marta", "Luis");
List<String> unicos = nombres.stream()
    .distinct()
    .toList();
System.out.println(unicos); // [Ana, Luis, Marta]
```

3.Interpretación

- Recorre el flujo
- Mantiene un conjunto interno de "ya vistos"
- Solo deja pasar los nuevos

4.Idea clave (conexión con Set / equals/hashCode)

Si trabajas con objetos propios (Persona, Producto…), `distinct()` funciona bien solo si el contrato `equals/hashCode` está bien implementado.

4) Equivalencia imperativa

```
List<String> unicos = new ArrayList<>();
Set<String> vistos = new HashSet<>();

for (String s : nombres) {
    if (!vistos.contains(s)) {
        vistos.add(s);
        unicos.add(s);
    }
}
```

`distinct()` encapsula este patrón.

5) Observación crítica (muy importante)

Si trabajas con objetos propios:

```
class Persona { String nombre; int edad; }
```

`distinct()` solo funciona correctamente si:

- `equals()` está bien implementado
- `hashCode()` es coherente con equals

Conecta directamente con el bloque de `HashSet` visto antes.

6) Complejidad

Tiempo aproximado: O(n)
Memoria adicional: O(n)

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Intermedias.

limit(): restricción del flujo y short-circuiting.

1) Definición

`limit(n)` es una operación intermedia.

- Deja pasar como máximo `n` elementos.
- Es una operación de corte temprano (*short-circuiting*).

2) Ejemplo básico

```
import java.util.List;
List<Integer> datos = List.of(5, 2, 9, 1, 4, 8);
List<Integer> primeros = datos.stream()
    .limit(3)
    .toList();
System.out.println(primeros); // [5, 2, 9]
```

3) Ejemplo combinado real

```
List<Integer> resultado = datos.stream()
    .sorted()
    .limit(3)
    .toList();
System.out.println(resultado); // [1, 2, 4]
```

"Ordena y quédate con los 3 primeros".

4) Interpretación técnica

El pipeline se ejecuta elemento a elemento.
Cuando se han producido `n` resultados:
→ el Stream se detiene
→ no procesa el resto
Esto no ocurre con todas las intermedias.

5) Equivalencia imperativa

```
List<Integer> primeros = new ArrayList<>();
int contador = 0;
for (Integer x : datos) {
    if (contador == 3) break;
    primeros.add(x);
    contador++;
}
```

`limit()` encapsula este patrón.

6) Complejidad

- En muchos casos puede evitar recorrer toda la colección.
- Especialmente útil con Streams grandes o infinitos.

Idea clave

`limit()` no transforma datos.
Controla la cantidad de resultados.

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Terminales y Reducción.

Operaciones Terminales: Reducción y Materialización.

Las operaciones terminales **cierran el Stream** y producen un resultado.
Sin una operación terminal, el pipeline **no se ejecuta**.

1) Tipos de operaciones terminales

- **Agregación**: `sum, count, max, min`
- **Reducción general**: `reduce`
- **Materialización**: `collect`
- **Acción final**: `forEach`

2) Ejemplo: reducción con `reduce`

```
import java.util.List;

List<Integer> datos = List.of(1, 2, 3, 4);
int suma = datos.stream()
        .reduce(0, (acum, x) -> acum + x);
System.out.println(suma);
```

Qué está ocurriendo

- `0` → valor inicial
- `(acum, x)` → acumulador
- Se combina cada elemento con el resultado parcial

Equivalente imperativo

```
int suma = 0;
for (int x : datos) {
    suma = suma + x;
}
```

`reduce` generaliza este patrón

3) Ejemplo: materialización con `collect`

```
import java.util.List;
import java.util.stream.Collectors;

List<Integer> resultado = datos.stream()
        .filter(x -> x % 2 == 0)
        .collect(Collectors.toList());
```

Qué está ocurriendo

- Se ejecuta el pipeline
- Se construye una nueva colección
- No se modifica la original

4) Diferencia clave

Operación	Resultado
reduce	valor único
collect	colección
forEach	efecto lateral

5) Interpretación técnica

- `reduce` → combina elementos en un único resultado
- `collect` → transforma el flujo en una estructura concreta

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Terminales y Reducción.

Operaciones Terminales: Collectors() vs toList().

En **Java Streams** existen dos formas habituales de materializar un flujo (`Stream`) en una colección `List`.

1. Uso de `toList()` (Java 16+)

Desde **Java 16** el interfaz `Stream` incorpora el método terminal:

```
List<String> resultado = stream
        .filter(s -> s.length() > 3)
        .map(String::toUpperCase)
        .toList();
```

Características importantes:

- Es **operación terminal**.
- No requiere `Collectors`.
- Devuelve una **lista no modificable (unmodifiable)**.
- Es más **conciso y legible**.

Ejemplo:

```
List<Integer> numeros = Stream.of(1,2,3,4,5)
        .filter(n -> n % 2 == 0)
        .toList();
```

Resultado: `[2,4]`

Pero si haces: `numeros.add(6);`

obtendrás: `UnsupportedOperationException` porque la lista es **inmutable estructuralmente**.

`toList()` es simplemente un **atajo especializado**.

2. Uso de `collect(Collectors.toList())`

Antes de Java 16, la forma habitual era:

```
List<String> resultado = stream
        .filter(s -> s.length() > 3)
        .map(String::toUpperCase)
        .collect(Collectors.toList());
```

Características:

- También es **operación terminal**.
- Usa el mecanismo general **Collector**.
- Devuelve una **lista modificable** (normalmente `ArrayList`).
- Permite usar otros colectores más complejos.

Ejemplo:

```
List<Integer> numeros = Stream.of(1,2,3,4,5)
        .filter(n -> n % 2 == 0)
        .collect(Collectors.toList());
numeros.add(6);   // válido
```

3. Diferencia conceptual

`collect()` es un **mecanismo general de reducción mutable** del stream.

Es decir: `Stream → Collector → estructura final`

Por ejemplo:

```
.collect(Collectors.toSet())
.collect(Collectors.groupingBy(...))
.collect(Collectors.joining())
.collect(Collectors.toMap(...)
```

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Terminales y Reducción.

Operaciones Terminales: Agregación y Acción Final.

Las operaciones terminales **activan el pipeline** y producen un resultado o un efecto final.

1) Agregación numérica (especialización de tipos)

```
import java.util.List;
List<Integer> datos = List.of(1, 2, 3, 4, 5, 6);
int suma = datos.stream()
        .filter(x -> x % 2 == 0)
        .mapToInt(x -> x)
        .sum();
```

Qué está ocurriendo

- `mapToInt` convierte el flujo a `IntStream`
- `sum()` realiza la agregación eficiente

Especialización para tipos primitivos (evita boxing)

2) Agregación general con `reduce`

```
int suma = datos.stream()
        .reduce(0, (acum, x) -> acum + x);
```

Interpretación

```
int acum = 0;
for (int x : datos) {
    acum = acum + x;
}
```

`reduce` es el patrón general de acumulación

3) Acción final con `forEach` (lambda)

```
datos.stream()
      .filter(x -> x % 2 == 0)
      .forEach(x -> System.out.println(x));
```

Qué representa `forEach`

- Operación terminal
- Ejecuta una acción sobre cada elemento
- No devuelve resultado

4) Acción final con referencia a método (: :)

```
datos.stream()
      .filter(x -> x % 2 == 0)
      .forEach(System.out::println);
```

Qué significa `System.out::println`

Equivale a:

```
x -> System.out.println(x)
```

Es una **referencia a método**

Las referencias a método (: :) se usan **cuando una lambda solo llama a un método existente**. No aportan funcionalidad nueva: son una **forma más concisa y legible** de escribir exactamente lo mismo.

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Streams: Operaciones Terminales y Reducción.

Operaciones Terminales: Agregación y Acción Final: Count().

1) Definición

`count()` es una operación terminal.

- Devuelve un `long`
- Cuenta los elementos que llegan al final del pipeline

Activa la ejecución del Stream.

2) Ejemplo directo

```
import java.util.List;
List<Integer> datos = List.of(1, 2, 3, 4, 5, 6);
long total = datos.stream().count();
System.out.println(total); // 6
```

3) Ejemplo con filtro

```
long pares = datos.stream()
    .filter(x -> x % 2 == 0)
    .count();
System.out.println(pares); // 3
```

Interpretación:

No cuenta la colección original.
Cuenta el resultado del pipeline.

4) Equivalencia imperativa

```
long pares = 0;
for (int x : datos) {
    if (x % 2 == 0) {
        pares++;
    }
}
```

`count()` encapsula este patrón.

5) Relación con otras terminales

Operación	Devuelve
count	cantidad
sum	valor numérico
reduce	valor acumulado
collect	colección

6) Observación técnica

- Devuelve `long`, no `int`.
- Es más expresivo que `reduce(0, ...)` para contar.

Idea clave

`count()` es la forma declarativa del patrón “contador en bucle”.

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Referencias a Métodos.

Referencias a método (::): simplificación de lambdas.

Las **referencias a método** permiten sustituir una lambda cuando esta **únicamente invoca un método existente**.

1) Regla fundamental

Si una lambda se limita a llamar a un método, puede sustituirse por una referencia a método.

2) Ejemplo básico (Consumer — API Java)

Interfaz: `void accept(T t);`
Lambda: `x -> System.out.println(x)`
Referencia a método: `System.out::println`
Uso: `datos.stream().forEach(System.out::println);`

3) Ejemplo de transformación (Function — API Java)

Interfaz: `R apply(T t);`
Lambda: `s -> s.length()`
Referencia: `String::length`
Uso: `datos.stream().map(String::length);`

4) Ejemplo de comparación (Comparator — API Java)

Interfaz: `int compare(T o1, T o2);`
Lambda `(a, b) -> a.compareTo(b)`
Referencia: `String::compareTo`

5) Tipos de referencias a método

Tipo	Forma	Ejemplo
Método de instancia	`obj::metodo`	`System.out::println`
Método estático	`Clase::metodo`	`Integer::parseInt`
Método de instancia (tipo)	`Clase::metodo`	`String::length`

6) Cuándo NO usar : :

Cuando hay lógica:

```
x -> x * 2
```

Cuando hay varias instrucciones:

```
x -> {
    int y = x * 2;
    return y;
}
```

7) Equivalencia conceptual

`System.out::println`
equivale a:

```
x -> System.out.println(x)
```

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Operaciones Terminales avanzadas y Optional.

Búsquedas, Máximos y Optional.

Hasta ahora hemos visto agregación y acción final.
Falta un tipo clave de operaciones terminales: **búsqueda y selección**, que introducen el uso de **Optional**.

1) Problema

¿Qué ocurre si buscamos un elemento… y no existe?
No siempre hay resultado
No se puede devolver "null" sin riesgo

2) `findFirst()` — Búsqueda

```
import java.util.List;
List<Integer> datos = List.of(1, 3, 5, 8, 9);
var resultado = datos.stream()
        .filter(x -> x % 2 == 0)
        .findFirst();
System.out.println(resultado);
```

Qué devuelve

No devuelve un `int`
Devuelve un **Optional<Integer>**

3) ¿Qué es Optional?

Un contenedor que puede tener valor… o no tenerlo

4) Uso de `orElse`

```
int valor = datos.stream()
        .filter(x -> x % 2 == 0)
        .findFirst()
        .orElse(-1);
```

Interpretación

- Si encuentra → devuelve el valor
- Si no encuentra → devuelve `-1`

5) `max()` y `min()`

```
var max = datos.stream().max(Integer::compareTo);
var min = datos.stream().min(Integer::compareTo);
```

También devuelven Optional, para evitarlo:

```
int maximo = datos.stream()
        .max(Integer::compareTo)
        .orElse(0);
```

6) Tipos de operaciones terminales (visión completa)

Tipo	Ejemplo	Resultado
Agregación	`sum, reduce`	valor
Materialización	`toList`	colección
Acción	`forEach`	efecto
Búsqueda	`findFirst`	Optional
Selección	`max, min`	Optional

Sin operación terminal → no hay ejecución

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Collectors: Materialización y transformación avanzada.

Collectors: Tranformación de Streams en Estructuras.

Los `Collectors` permiten transformar un Stream en una **estructura de datos final** o en un **resultado más complejo**.

1) ¿Qué hace `collect()`?

```
Stream → Collector → Resultado
```

Es una operación terminal

Construye el resultado final

2) Ejemplo básico

```
import java.util.stream.Collectors;
List<Integer> resultado = datos.stream()
        .filter(x -> x % 2 == 0)
        .collect(Collectors.toList());
```

Interpretación

- Se ejecuta el pipeline
- Se construye una lista nueva
- No se modifica la original

3) Ejemplo: agrupación (muy importante)

```
import java.util.Map;
import java.util.stream.Collectors;

Map<Boolean, List<Integer>> agrupado =
        datos.stream()
             .collect(Collectors.groupingBy(x -> x % 2 == 0));
```

Resultado

```
true  → [pares]
false → [impares]
```

4) Ejemplo: conteo por categoría

```
Map<Boolean, Long> conteo =
        datos.stream()
             .collect(Collectors.groupingBy(
                 x -> x % 2 == 0,
                 Collectors.counting()
             ));
```

5) Tipos de Collectors (nivel DAW)

Tipo	Uso
`toList()`	lista
`toSet()`	conjunto
`groupingBy()`	agrupación
`counting()`	conteo

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Integración Final.

Modelo Completo: de colección a resultado.

Esta diapositiva integra todo lo visto: **colecciones + lambdas + interfaces funcionales + streams + operaciones terminales**.

1) Esquema completo del procesamiento

```
Colección →Stream →Operaciones(lambda) →Resultado
```

Más detallado:

```
fuente →filter →map →(otras operaciones) →terminal
```

2) Ejemplo completo integrado

```
import java.util.List;
List<Integer> datos = List.of(1, 2, 3, 4, 5, 6);
int resultado = datos.stream()
        .filter(x -> x % 2 == 0)  // condición
        .mapToInt(x -> x * x)     // transformación
        .sum();                   // agregación
System.out.println(resultado);
```

3) Descomposición técnica

- `stream()` → crea el flujo
- `filter` → aplica un `Predicate`
- `mapToInt` → aplica una transformación (`Function`)
- `sum` → operación terminal

4) Equivalencia imperativa

```
int resultado = 0;
for (int x : datos) {
    if (x % 2 == 0) {
        resultado += x * x;
}}
```

5) Qué se ha conseguido

Aspecto	Imperativo	Streams
Control	explícito	implícito
Código	detallado	declarativo
Reutilización	baja	alta
Legibilidad	media	alta

6) Elementos que intervienen

- **Colección** → almacena datos
- **Stream** → flujo de procesamiento
- **Lambda** → comportamiento
- **Interfaz funcional** → contrato
- **Operación terminal** → resultado

7) Lectura correcta del código

"De los datos, quédate con los pares, transforma al cuadrado y suma el resultado."

Gestión de Datos en Memoria

Programación Funcional Sobre Colecciones.

Integración Final.

Streams – Checklist Mental de Consolidación.

1 Operaciones intermedias vs terminales

Intermedias → devuelven `Stream`

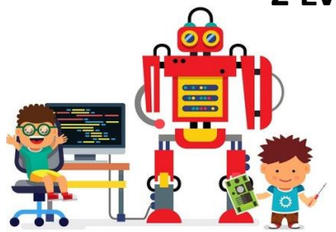
- filter
- map
- sorted
- distinct
- limit

Terminales → cierran el flujo

- collect
- reduce
- forEach
- count
- sum
- findFirst
- max / min

Sin operación terminal → no se ejecuta nada.

2 Evaluación perezosa (Lazy Evaluation)



- Las operaciones intermedias no ejecutan código por sí solas.
- El pipeline se activa solo al aparecer una operación terminal.
- No se crean colecciones intermedias.

Modelo correcto:

Cada elemento recorre todo el pipeline antes de pasar al siguiente.

3 Optional en búsquedas

Operaciones como:

- findFirst
- max
- min

Devuelven `Optional<T>`.

No devuelven null.

Debe gestionarse con:

- orElse
- orElseGet
- ifPresent

4 collect vs reduce

Operación	Resultado	Uso
reduce	valor único	acumulación
collect	colección / estructura	materialización
forEach	efecto	acción final

5 Streams no modifican la colección original

Salvo que exista efecto lateral explícito.

El flujo es una vista de procesamiento, no una estructura.

6 Modelo final integrado

`Colección → Stream → Operaciones → Terminal → Resultado`

Imperativo controla el flujo.

Streams describen la intención

TEMA 8:

ESTRUCTURAS DE ALMACENAMIENTO